City University London

MSc in Software Engineering with Cloud Computing

Project Report

2023-2024

Development of a Scalable Academic Opportunities Platform

David Enukashvili

Supervised by: Dr Vitaly Aksenov

Date of submission: 2nd October 2024

Declaration

By submitting this work, I declare that this work is entirely my own except those parts duly identified and referenced in my submission. It complies with any specified word limits and the requirements and regulations detailed in the coursework instructions and any other relevant programme and module documentation. In submitting this work I acknowledge that I have read and understood the regulations and code regarding academic misconduct, including that relating to plagiarism, as specified in the Programme Handbook. I also acknowledge that this work will be subject to a variety of checks for academic misconduct. Marks are provisional and subject to change in response to moderation, assessment board decisions and any ongoing investigations of suspected academic misconduct.

Signed: David Enukashvili

Abstract

This project focuses on the development of a scalable platform that allows the academic community to create and find opportunities like academic positions, internships, research cooperation, and others. Creation of this project is inspired by job search platforms, the aim is to create a cloud-based solution that addresses the fragmented nature of existing academic networks. Using Google Cloud Platform for scalability and security, the platform incorporates technologies like React for the frontend and Node.js for the backend. The results demonstrated that the platform achieves all the objectives and requirements it had to meet.

Keywords: academic platform, cloud computing, scalability, React, Node.js

Table of Contents

Declaration *2*

Abstract *3*

Chapter 1 – Introduction and Objectives *6*

1.1 Introduction **6**

1.2 Aim **7**

1.3 Project Objectives **7**

1.4 Purpose **8**

1.5 Work Plan **9**

1.6 Report Structure **9**

Chapter 2 – Context *11*

2.1 Overview **11**

2.2 Inspiration from Job Search Platforms (LinkedIn and Indeed) **11**

2.3 Academic Collaboration and the Role of Opportunities Platforms **12**

2.4 Challenges in Accessing Academic Opportunities **13**

2.5 Cloud Technology for Scalability **14**

2.6 Comparison of Existing Tools and Platforms **16**

2.7 The Need for Modernisation in the Academic World **17**

2.8 Conclusion **17**

Chapter 3 - Methods *18*

3.1 Technology stack **18**

3.1.1 Cloud Infrastructure 18

3.1.2 Frontend 18

3.1.3 Backend 19

3.1.4 Database 20

3.2 Architecture **20**

3.3 Environment setup **21**

3.4 Development **27**

Chapter 4 - Results *40*

4.1 Development result **40**

4.2 Testing **47**

Chapter 5 - Evaluation, Reflections, and Conclusions *48*

5.1 Results Evaluation and Reflection **48**

5.1.1 Comparing results to objectives 48

5.1.2 Problems encountered 49

5.1.3 Final Reflection 50

5.2 Conclusion **50**
5.2.1 Project conclusion 50
5.2.2 Possible further work 51
References ***52***
Appendix A – Project Proposal ***56***
Introduction **56**
Critical Context **57**
Approaches: Methods & Tools for Design, Analysis & Evaluation **59**
Literature Review 59
Analysis 59
Architecture 59
Technological Stack and Methodology **61**
Design Methodology 61
Technical Stack 61
Testing 64
Work plan **64**
Risks **65**
References **66**
Research Ethics Review Form for MSc Projects **68**
Appendix B – Source Code ***71***
B.1 Frontend **71**
B.2 Backend **102**

Chapter 1 – Introduction and Objectives

1.1 Introduction

The Academic Opportunities Platform enables members of the academic world to both create and find academic vacancies of various types, such as research collaborations, internships, and academic partnerships. The platform is essentially a project that functions as a centralized hub where all of these vacancies can be accessed by job seekers and posted by professors, institutions, or other organizations.

In recent years, the way people search and apply for jobs has changed dramatically with the emergence of professional platforms like LinkedIn and Indeed - these platforms have made it much easier to communicate with individuals and organizations, making it much easier for job seekers to search for vacancies using direct job postings, as well as to connect with professionals in the field. The platform concept I propose follows a similar logic, but instead of focusing exclusively on corporate vacancies, the project is designed specifically for the academic sector. The Academic Opportunities Platform allows users to post academic opportunities and collaborate on them. Such a platform has many advantages, one of the most important is that academic institutions and individuals often have difficulty finding suitable opportunities, as they are often scattered across different channels or networks. Thus, by developing a single scalable system, the Academic Opportunities Platform centralizes these opportunities, simplifying many processes and actions for students, researchers, and professors - studying, applying, and collaborating on various academic projects. Regardless of the academic background or field of

knowledge, the platform will be able to meet the needs of users, which is guaranteed by the scalability of the platform.

1.2 Aim

The primary aim of the project is to design and develop a scalable academic opportunities platform. Academic Opportunities Platform provides a prototype solution for connecting students, professors, researchers, and academic institutions by offering them a centralised space to post and discover various academic opportunities - by using cloud technology, the project is meant to build a system that can handle varying loads of users and offer a seamless experience, ensuring that the platform remains reliable even as its user base grows while being efficient with resource usage.

1.3 Project Objectives

The following objectives must be met to achieve the aim of this project.

1. **Design a Scalable Platform**: Design a flexible and scalable platform with the use of Cloud Technology to create a platform that will be able to perform consistently under different levels of load.
2. **Enable Sharing of Multidisciplinary Academic Opportunities**: It includes creating a platform for sharing many academic opportunities, starting from research projects to experimental activities for publication co-authorship and other types of opportunities
3. **Facilitate Collaboration**: Make mechanisms to facilitate the academic collaboration of different members of an academic community that wouldn't find each other without such a platform.

A list of requirements should be met to consider these objectives as met:

1. Objective 1 will be considered to be met if a developed platform uses cloud technology as the primary way to deploy it and store data used for its operation.
2. Objective 2 will be considered to be met if the platform has the functionality to publish different types of opportunities for all users that are registered on the said platform.
3. Objective 3 will be considered to be met if the platform has the functionality to establish communication on it in some ways (e.g. functionality to apply with motivational letters and communicate with candidates and opportunity creators).

1.4 Purpose

The purpose of this project is to address the need for a centralised hub where members of the academic community can post and apply to academic opportunities more efficiently. Inspired by the success of job-search platforms, this platform intends to modernise the way academic opportunities are distributed and accessed. Traditionally, academic opportunities such as research collaborations or internships are located across various institutions' networks, making it difficult for individuals outside those institutions to access them. This project seeks to eliminate these barriers by creating a scalable, cloud-based solution where opportunities from different academic disciplines are easily accessible. Ultimately, the purpose of the project is to promote academic collaboration and ensure that students, professors, and researchers can easily find relevant opportunities that align with their academic interests and career goals.

1.5 Work Plan

The project progress can be split into 5 phases shown in the following list with the allocated time proportion.

1. Review of literature and technologies that are necessary for the project (15%)
2. Initial project design (10%)
3. Environment setup (5%)
4. Development, testing, and deployment (55%)
5. Evaluation of results and writing the report (15%)

The first and second phases were partly concurrent. Phases 3 and 4 were started after the initial design was created and were done one after another. Phase 5 was started at the same time as phases 1 and 2 however, most of the work in this phase was done closer to the finish of the project.

1.6 Report Structure

The report is organized as follows:

- Chapter 1 - Introduction and Objectives

This chapter introduces the project and its aim and purpose but also defines the objectives of the project and the requirements to meet them.

- Chapter 2 - Context

This chapter provides a review of the literature and products that are similar to my project to set the context in which this project was conducted.

- Chapter 3 - Methods

This chapter reviews the different techniques studied during the project and presents the methods and processes used throughout the project.

- Chapter 4 - Results

This chapter provides the results of the development process as well as some analysis of the results.

- Chapter 5 - Evaluation, Reflections, and Conclusions

This chapter presents a global evaluation of the project, how it was conducted and problems encountered. It provides also a self-reflection about the knowledge gained and how the project would have been conducted differently. Further work is discussed too.

To not interfere with the reading of this report, all the substantial materials are provided in the appendix:

- Appendix A - Project Proposal is a copy of the project proposal submitted before the start of this project.
- Appendix B - Source code provides a copy of the source code used to run the experiments. It can also be found on a public GitHub repository at the following URL: https://github.com/DUVIED101/MScProject

Chapter 2 – Context

2.1 Overview

In the academic world today, digital platforms have to play a vital role in its functioning. However, these platforms are often unknown, old-fashioned, and not very useful. This section describes the reasons and inspirations for creating a platform that may fix these issues.

It also gives a little insight into the reasons to use cloud technologies when working on this project, as they are a cornerstone in this whole work.

2.2 Inspiration from Job Search Platforms (LinkedIn and Indeed)

Platforms like LinkedIn and Indeed have changed the way people look for jobs by enabling easy connections between employers and candidates all across the world. For instance, LinkedIn has more than 1 billion users, with over 300 million active monthly users globally, and it has become an essential tool for professionals in various sectors (Bondar, 2023). Another example, Indeed attracts around 350 million users per month (Indeed, 2024), making it another essential platform for all candidates and employers all over the world

There are a lot of factors that made these platforms as popular as they are nowadays. One of them is allowing searching for jobs in every imaginable field from any point on the map that has access to the internet. A UK survey found that 37% of adults used online platforms for job searching, with that number rising to 50% for those aged 16-44 (Ofcom, 2022). These numbers not only show how demand for platforms that help in finding a job, they also show that our society already depends heavily on what we have now.

Despite the fact world of academic opportunities is by definition smaller than the job market, it still needs a similar instrument in my opinion. So, much like LinkedIn and Indeed, the platform proposed in this project will provide a

space where users can post and find academic opportunities, including research positions, collaborations, and internships. The goal was to make these opportunities as easy to access as possible for users, to get as close as the scope of the projects allows to the experience job seekers feel when searching for it.

2.3 Academic Collaboration and the Role of Opportunities Platforms

In the academic world, collaboration is key to innovation and success. Several studies consistently show that collaborative work leads to higher research output and has a broader impact than work completed in isolation (Wuchty, Jones, and Uzzi, 2007). However, there is not always an obvious way to find opportunities that suit a certain candidate, especially considering geographical constraints. They are present because, in many cases, collaboration arises from networking, personal connections, or institutional partnerships, but these traditional pathways are not always accessible to everyone.

Several universities and research institutions already offer some tools to promote collaboration in this area. For example, Oxford University provides a list of research collaboration tools (Oxford IT Services, 2024), and Harvard University offers software to support research collaboration (Harvard Data Management, 2024). While these tools are useful, they are very specific in the provided functionality, as well as often being used within specific institutions or fields of study, making it hard for users to find opportunities outside of these institutions.

Other platforms, such as arXiv, offer a repository for sharing preprints and research but don’t address the broader need for a centralised academic opportunities platform that allows the creation of new opportunities and finding candidates for them across different institutions (arXiv, 2024). In

contrast, the developed platform will allow academics to find and engage with a wider variety of collaborative opportunities, from internships and co-authorships to larger projects either on-site at a specific institution or completely remote. This approach democratises access to research opportunities, enabling users to build connections that might otherwise be difficult to establish.

2.4 Challenges in Accessing Academic Opportunities

One of the primary barriers facing students and researchers today is the way academic opportunities are published nowadays. These opportunities are often scattered across various institutional websites, job boards, or internal networks. Sometimes, they are not posted anywhere, and candidates are found through personal connections. This practice makes it difficult to find and access them. Furthermore, many academic job boards focus primarily on faculty positions, leaving out opportunities for internships, co-authorship, or research collaborations.

This fragmentation not only makes it difficult for individuals to discover relevant opportunities but also creates a vicious circle in which people without connections or direct access to internal job boards will never get those connections due to a lack of experience in collaboration. Early-career researchers, for example, may not have the networks or resources to find valuable opportunities, particularly when these are shared informally or through closed institutional channels.

The platform created during the project aims to address this issue by offering a centralised, scalable system where academic opportunities from a wide range of institutions can be posted and accessed. By providing a more transparent and user-friendly interface, this platform could help level the playing field, ensuring that more people—regardless of their background or

connections—can find and apply for relevant academic opportunities. Platforms like AcademicPositions and UKRI provide some of these services, but they tend to focus primarily on job postings or research grants, leaving out broader categories of academic engagement (AcademicPositions, 2024; UKRI, 2024).

2.5 Cloud Technology for Scalability

When designing for any platform that is meant to serve a significant user base, one must focus heavily on ensuring scalability, i.e. it should be able to scale and shrink depending on demand whilst never sacrificing reliability or performance. And the role of cloud technology cannot be more prominent in enabling us to meet those goals as well. With this adoption of a fully online platform, we will be able to leverage the flexibility and scalability offered by cloud computing directly on the Google Cloud Platform (GCP), guaranteeing that the platform is designed for use across many academic settings. With the broadest range of services that could be dynamically allocated, it meant that when there were spikes in usage from so many different regions during a live game, those systems would expand or contract to ensure all users get minimum latency and optimal performance under varying loads GCP (Google Cloud,2024).

Scalability is particularly important in academic settings due to the cyclical nature of usage. For example, academic platforms may experience peak traffic during certain times of the year, such as when universities open application windows for internships or research programs or during conference seasons when many candidates may apply. At other times, usage may be lower, such as during semester breaks or off-peak research periods. With GCP, the platform can handle this change of traffic seamlessly. It ensures that when user activity spikes, additional resources are automatically allocated to

maintain the platform's performance without the need for manual intervention (Chen et al., 2019). Conversely, when traffic decreases, the system can scale back its resource usage, optimising cost-efficiency without sacrificing user experience.

Security is another important thing to take into consideration when designing a platform for academic use, as it often involves sensitive data related to research, personal information, and institutional details. Google Cloud Platform provides security features, such as encryption for data at every stage, identity and access management (IAM) system, and protection against Distributed Denial of Service (DDoS) attacks (Google Cloud, 2024). These features are particularly important in the academic sector, where data leaks can have serious consequences, like violating privacy laws like the General Data Protection Regulation (GDPR) in the European Union (Bui et al., 2021).

In addition to scalability and security, the cloud-based nature of the platform ensures that it can evolve over time. As the platform grows, it can integrate new features, tools, and services without the need for major infrastructure overhauls. This adaptability is essential in the academic world, where technological trends and user needs are constantly shifting. Cloud technology allows the platform to stay relevant by accommodating new developments in academic collaboration, data management, and research workflows (Fernández et al., 2018).

Furthermore, GCP supports the integration of machine learning and artificial intelligence (AI) tools, which could be leveraged to enhance the platform's capabilities in the future. For instance, AI-driven recommendation systems could help users discover relevant academic opportunities based on their profiles and previous activities, further enhancing user engagement (Zhao and Liu, 2020). While this particular functionality was not added during the

development process, the platform’s ability to seamlessly integrate these advanced technologies is another benefit of adopting cloud infrastructure, positioning it to evolve alongside technical advancement.

In summary, GCP's cloud technology forms the backbone of this academic opportunities platform, providing the scalability, security, and adaptability necessary to serve a growing and dynamic user base. As the platform expands and the academic landscape continues to evolve, cloud technology ensures that it remains a reliable, high-performing, and future-proof solution.

2.6 Comparison of Existing Tools and Platforms

Several existing platforms address specific needs within academia, but none offer the comprehensive scope required to meet the diverse needs of the global academic community. For example, Konfer focuses on connecting universities and businesses for collaborative projects but doesn’t provide a wide range of opportunities for students and researchers (Konfer, 2024). Similarly, AcademicPositions offers job listings but lacks options for internships, research partnerships, or other types of academic collaborations (AcademicPositions, 2024).

UKRI, which primarily supports funding and grant opportunities, also offers valuable resources but doesn’t cover other academic opportunities, such as internships or cross-institutional research collaborations (UKRI, 2024). The platform proposed in this project seeks to combine the strengths of these existing tools while addressing their limitations, offering a more comprehensive and user-friendly solution that serves students, researchers, and institutions alike.

2.7 The Need for Modernisation in the Academic World

The academic world has been slower than the corporate sector to adopt centralised, scalable digital platforms. While LinkedIn and Indeed have dramatically improved how professionals connect with employers and find jobs, academic opportunities remain scattered across a variety of disconnected sources. This creates inefficiencies that can prevent researchers, students, and institutions from finding the best opportunities available.

Modernising the academic space with a centralised platform will simplify how people engage with the academic world. By leveraging proven models from job search platforms and integrating advanced cloud technologies, the platform aims to transform the academic landscape, making it easier for students, researchers, and professors to connect and pursue collaborative opportunities. This modernisation is crucial for ensuring that the academic community remains competitive and continues to attract top talent from around the world.

2.8 Conclusion

In conclusion, the critical context of this project highlights the need for a scalable, user-friendly platform that addresses the challenges faced by the academic community in accessing opportunities. Drawing inspiration from successful job search platforms like LinkedIn and Indeed, this project aims to provide a centralised platform that enhances collaboration and broadens access to academic opportunities. By utilising cloud technology, the platform will ensure scalability, reliability, and security, making it an essential tool for students, researchers, and academic institutions worldwide. Ultimately, this platform has the potential to reshape the academic landscape, fostering greater collaboration and making academic opportunities more accessible to all.

Chapter 3 - Methods

3.1 Technology stack

Before explaining how the development process was conducted, I decided to dedicate a little part to an overview of the technology stack that was used during the work on the project.

3.1.1 Cloud Infrastructure

For this project, I selected Google Cloud Platform (GCP) due to its scalability, reliability, and variety of tools that can be used in cloud-based development. The general requirement for the platform is its ability to scale depending on changes in user traffic, particularly during high-demand periods. GCP's architecture ensures that resources can dynamically adjust to these variations, maintaining performance under both light and heavy loads.

The greatest advantage of GCP is its set of integrated tools - GCP offers built-in services for computing, storage, networking, and deploying projects. It helps to align perfectly with the needs of a scalable academic opportunities platform. In particular, GCP integration with Google Identity Platform and Google Identity and Access Management enables secure authentication and authorisation of users, ensuring high levels of privacy protection.

The platform's choice of GCP was further influenced by its containerisation capabilities, especially through Google App Engine - a fundamental component for ensuring scalability in my project.

3.1.2 Frontend

React was chosen as a frontend framework due to its efficient component architecture - it allows to create scalable applications. Since the project uses a microservice architecture, which will be described later in this report, React's important ability to split the interface into reusable

components ensures the independence of the platform sections from each other; they can be developed and supported as independent units. The React ecosystem allows to significantly speed up the development process due to a wide range of available ready-made libraries and components - the framework provides access to both popular libraries like Axios and access to some exclusive libraries, using which some application functions were developed, for example, React Router and some testing libraries. In addition to speeding up the development process, the extensive ecosystem of the framework, React, allows the use of reliable solutions for common interface problems. Also, the possibility of seamless integration with other technologies used in the project (internal APIs and cloud services) made React the best choice for creating an Academic Opportunities Platform.

3.1.3 Backend

Node.js was chosen for backend development because it is particularly suitable for applications that expect a high level of concurrent user connections. It has an event-driven, non-blocking architecture. Users posting jobs, browsing lists, and submitting applications - the platform must handle multiple simultaneous interactions. Node.js is extremely efficient in environments with a large number of parallel processes, it can handle multiple requests without degrading performance.

An important detail in the context of the entire project is the use of Node.js for backend development, which allows the use of a single programming language for both the frontend and backend - JavaScript. In addition, the need for real-time communication between users and the platform is implemented using Node.js, namely the technology for handling asynchronous operations, which is necessary to ensure that the platform always remains responsive under high user load.

3.1.4 Database

Google Cloud Database Spanner was chosen as the appropriate database solution for this project because it combines the consistency of traditional relational databases with the scalability of NoSQL systems. Google Cloud Spanner enables efficient management of complex relationships between academic opportunities, user data, and interactions. In addition, Spanner's ability to execute large-scale SQL queries makes it an optimal choice without breaking the strong consistency of a globally distributed database.
The database must handle complex transactions, such as when users apply for multiple positions or when institutions publish complex listings. Globally, Spanner's architecture supports these operations smoothly, even with large transaction volumes.

Spanner's distributed nature, as well as its automatic scaling, together enable it to manage the expected growth of the platform - both horizontal scalability and strong transaction support are provided. Moreover, Spanner guarantees a steady data flow of the various components of the platform due to its integration with other Google Cloud services.

3.2 Architecture

For this project, there was a choice made to use microservice architecture. This allowed to development of different parts of the application, especially the backend, mostly independent from one another. While the frontend was developed inside one application, the backend part of the project was divided into four services:

1. User service – this service was created to control user authorisation and authentication as it created authentication tokens for other services to operate properly.

2. Opportunity service – this service manages everything connected to the opportunities themselves: opportunity creation, editing and deletion, as well as applications made to the posted opportunities.
3. Search service – this service manages searching and filtering the opportunities for candidates who wish to apply to them. While this service gives only basic functionality at this stage, it may be developed into a powerful search engine, which was not quite possible considering the time allocated for the project.
4. Messaging service – this service has a simple conversation functionality, including creating conversations and storing all the messages sent.

Services 1, 2 and 4 have their databases to manage data that is needed for their operation, while the Search service uses the opportunity services' database as it is only used to read data, not to write anything.

The usage of microservices architecture helped to divide and sometimes parallelise the development process. It also provides an opportunity to easily update the functionality of each service in future, like adding a recommendation system to the search engine, without a risk of harming other parts of the platform.

3.3 Environment setup

This part explains how the development environment was set up to deploy the project on the cloud.

The development process is started by setting up the cloud environment. As mentioned before, for this project, I used Google Cloud, so firstly, a Google account is needed to create a Google Cloud project. Because some GCP services are not free, the 90-day trial was activated to get the necessary credits to use them for the project.

For this project to operate, these services were mainly used:

1. Google Cloud Spanner - A scalable, fully managed relational database service used for storing data in databases.
2. Google App Engine (GAE) – a Platform that allows you to build and deploy applications without having to manage the underlying infrastructure.
3. Google Cloud IAM (Identity and Access Management) - Used for managing access control to GCP services, including assigning permissions for various roles and services.
4. Google Cloud Storage - Used for storing static files, logs, and other artifacts related to the project.
5. Google Cloud Logging - Provides monitoring and debugging services for logging events and errors occurring within the system.
6. Google Cloud Identity - Used for authentication and user management across the system for secure access.
7. Google Cloud Build - A continuous integration and delivery (CI/CD) service used for building, testing, and deploying the application.
8. Google Cloud SQL - A fully-managed relational database service used for specific SQL-based storage needs that are separate from Spanner.
9. Google Cloud Identity - Used for authentication and user management across the system for secure access.

While usage of these services should be enough for the project to operate properly, here is a list of every single API and service that was activated before and during the development process:

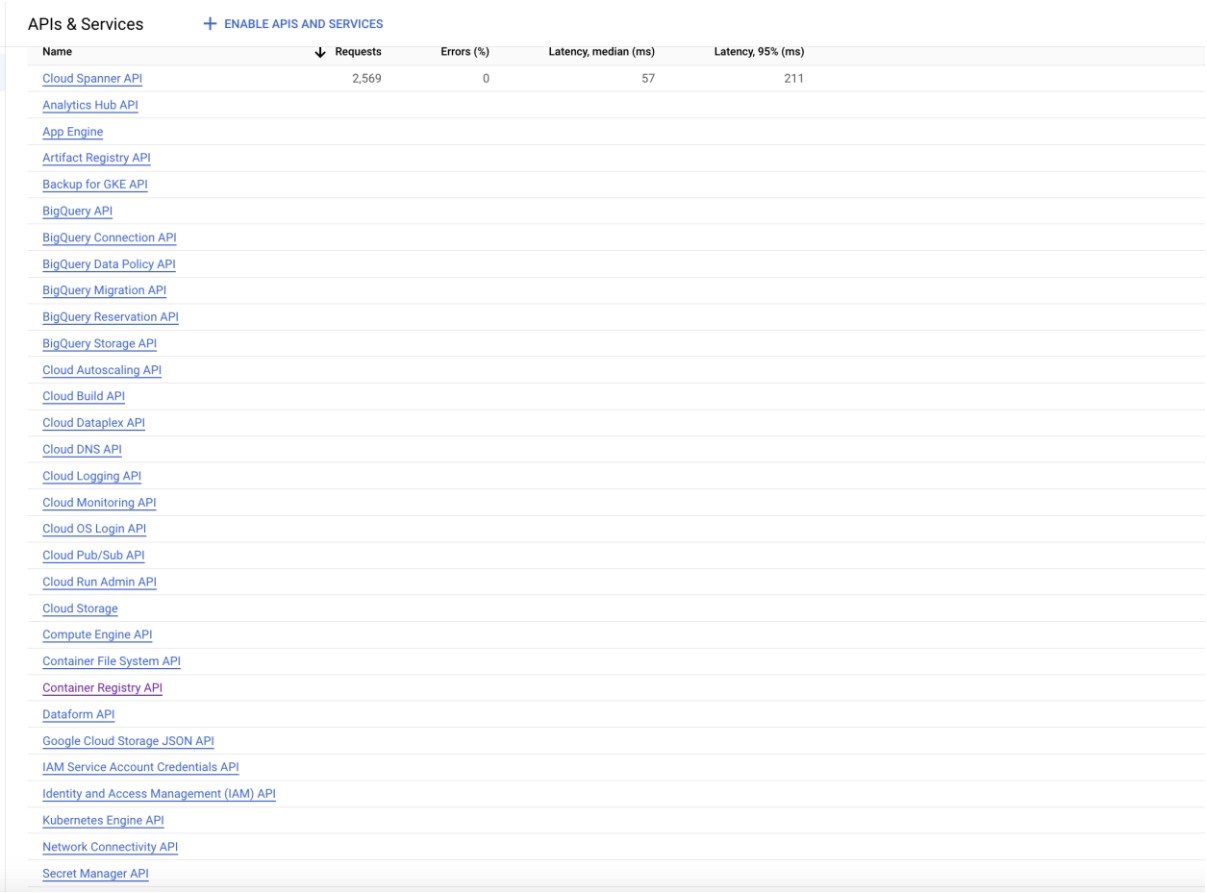


Figure 1 - All enabled APIs and Services

The next step is creating a Service Account (Located in "IAM & Admin" > "Service Accounts."). For this account to work, there is a list of roles that it should have:

1. Cloud Spanner Admin
2. Kubernetes Engine Admin
3. Cloud Build Editor
4. Storage Admin
5. Viewer
6. Compute Admin
7. Monitoring Viewer
8. Logging Viewer
9. Service Account User

The next step is setting up a database instance in Google Spanner.

For this project, there was created one instance on Google Spanner and a few databases inside it.

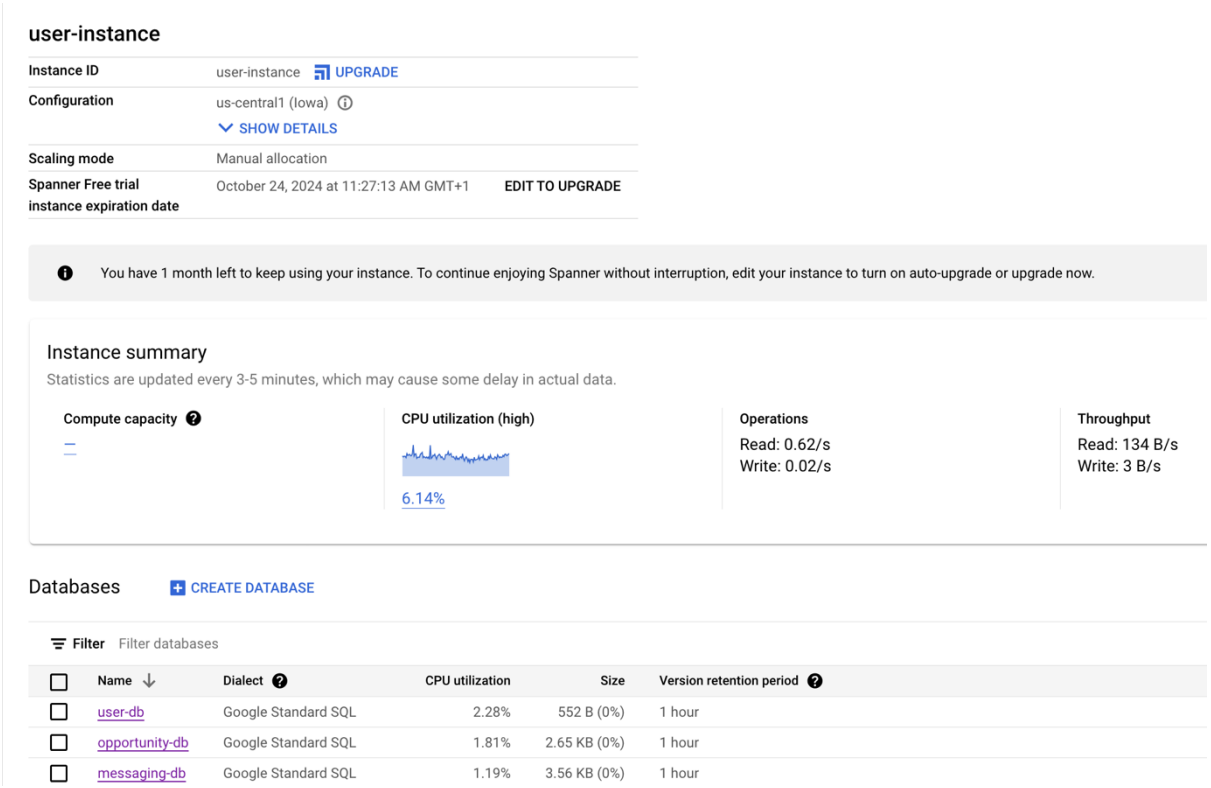


Figure 2-Project Instance with Databases

It was mistakenly called ‘user-instance’, however, this instance is used by every service.

In user database, only one table is created to store user data, and the DDL statement for it is:

```
CREATE TABLE users (
  id STRING(36) NOT NULL,
  name STRING(256),
  email STRING(256) NOT NULL,
  password STRING(256),
  googleId STRING(256),
) PRIMARY KEY(id);;
```

In the opportunity database, there are two tables: opportunities and applications, their DDL statements are:

For Opportuinities table

```
CREATE TABLE Opportunities (
  OpportunityID STRING(36) NOT NULL,
  Title STRING(MAX) NOT NULL,
  Description STRING(MAX),
  Location STRING(MAX),
  Deadline DATE,
  PostedBy STRING(36) NOT NULL,
  EducationLevel STRING(MAX),
  SubjectFilters ARRAY<STRING(MAX)>,
) PRIMARY KEY(OpportunityID);;
```

For Applications table

```
CREATE TABLE Applications (
  OpportunityID STRING(36) NOT NULL,
  ApplicationID STRING(36) NOT NULL,
  UserID STRING(36) NOT NULL,
  ApplicationDate TIMESTAMP NOT NULL OPTIONS (
    allow_commit_timestamp = true
  ),
  MotivationLetter STRING(MAX),
  CVUrl STRING(MAX),
) PRIMARY KEY(OpportunityID, ApplicationID),
  INTERLEAVE IN PARENT Opportunities ON DELETE CASCADE;;
```

And the last database, Messaging database, has two tables: Conversations and Messages.

DDL for Conversations table:

```
CREATE TABLE Conversations (
  ConversationID STRING(36) NOT NULL,
  OpportunityID STRING(36) NOT NULL,
  CreatorID STRING(36) NOT NULL,
  ApplicantID STRING(36) NOT NULL,
  LastMessageTimestamp TIMESTAMP,
) PRIMARY KEY(ConversationID);;
```

DDL for Messages table:

```
CREATE TABLE Messages (
  MessageID STRING(36) NOT NULL,
  ConversationID STRING(36) NOT NULL,
  SenderID STRING(36) NOT NULL,
  MessageContent STRING(MAX) NOT NULL,
  Timestamp TIMESTAMP NOT NULL,
) PRIMARY KEY(ConversationID, MessageID),
  INTERLEAVE IN PARENT Conversations ON DELETE CASCADE;;
```

The last thing to set up on the Google Cloud is a storage bucket for CV files. It is created in the Cloud Storage service.

After setting up everything on a GCP side, it is important to put some environmental variables so each service and frontend application can access everything created on Google Cloud.

The following code snippets were taken from .env files, they have different variables used in the project and their purpose is explained in more depth in the Development chapter.

.env for the frontend app:

```
REACT_APP_BACKEND_USER_URL=USER-SERVICE-URL
REACT_APP_GOOGLE_CLIENT_ID=GOOGLE-CLIENT-ID(for auth purposes)
REACT_APP_BACKEND_OPPORTUNITY_URL=OPPORTUNITY-SERVICE-URL
REACT_APP_BACKEND_SEARCH_URL= SEARCH-SERVICE-URL
REACT_APP_BACKEND_MESSAGING_URL= MESSAGING-SERVICE-URL
```

.env for the User service:

```
PORT=5001
JWT_SECRET=jwt-secret
GOOGLE_CLIENT_ID=client-id
GOOGLE_CLIENT_SECRET=google-client-secret
SPANNER_PROJECT_ID=project-id
SPANNER_INSTANCE_ID=instance-id
SPANNER_DATABASE_ID=user-database-id
```

.env for the Opportunity service:

```
PORT=5002
JWT_SECRET=jwt-secret
USER_SERVICE_URL=user-service-url

SPANNER_PROJECT_ID=project-id
SPANNER_INSTANCE_ID=instance-id
SPANNER_DATABASE_ID=opportuinity-database-id
BUCKET_NAME=bucket-name (to store cvs)
```

.env for the Search service

```
PORT=5003
```

```
JWT_SECRET=jwt-secret
USER_SERVICE_URL=user-service-url

SPANNER_PROJECT_ID=project-id
SPANNER_INSTANCE_ID=instance-id
SPANNER_DATABASE_ID=opportunity-database-id
```

.env for the messaging service

```
PORT=5004
SPANNER_PROJECT_ID=project-id
SPANNER_INSTANCE_ID=instance-id
SPANNER_DATABASE_ID=messaging-database-id
SPANNER_USERS_DATABASE_ID=user-database-id
SPANNER_OPPORTUNITY_DATABASE_ID=opportunity-database-id
USER_SERVICE_URL=user-service-url
JWT_SECRET=jwt-secret
SMTP_HOST=smtp.gmail.com
SMTP_PORT=465
SMTP_USER=service-email-address
SMTP_PASS=app-password (created in the gmail account)
```

These are necessary steps to prepare the project for deployment to the Google App Engine. The deployment is done using gcloud shell in the terminal, and is configured with app.yaml files (they are described in the Development chapter).

3.4 Development

This part of the report covers the development process, including, project structure, mainly used libraries, description of some used functions, etc.

To keep this report from growing to an unreadable size, the functions that will be shown and described in this chapter are the ones that are essential for providing a certain functionality, and they may be considered highlights of this project. All the code written for the project, with some commentary, is present in Appendix B.

This project was developed using microservices architecture, so the whole structure of the project looks like:

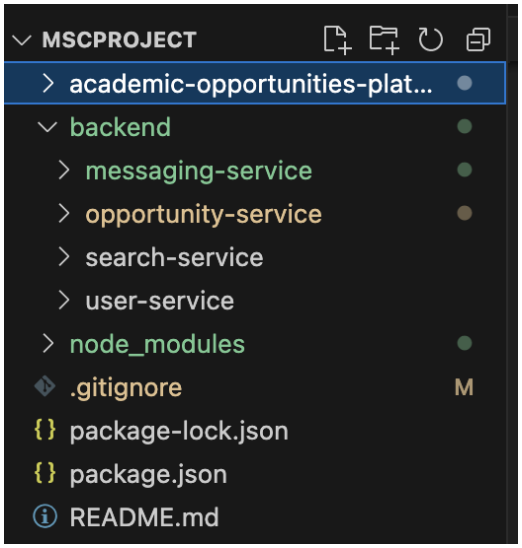


Figure 3 - Project Structure in Visual Studio Code

In Figure 2 it can be seen that Backend has 4 different folders, each of them containing one service, while “academic-opportunities-platform” is a folder where the react application is stored.

Taking the user service as an example, let’s look into the structure of the service itself.

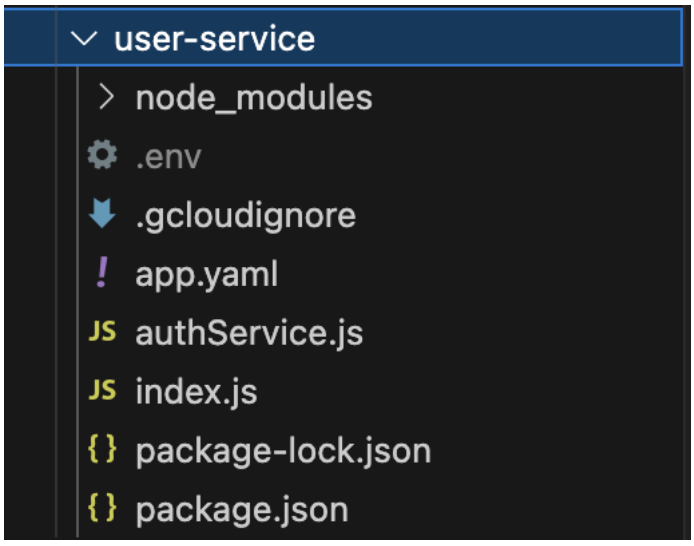


Figure 4-Microservice structure

Most of the backend logic is always in the index.js, but before looking at it, it is important to look in configurational files, starting with .env - file where several variables are stored:

```
PORT=5001
JWT_SECRET=jwt-secret
GOOGLE_CLIENT_ID=client-id
GOOGLE_CLIENT_SECRET=google-client-secret
SPANNER_PROJECT_ID=project-id
SPANNER_INSTANCE_ID=instance-id
SPANNER_DATABASE_ID=database-id
```

JWT secret is needed for token verification, Google Client ID and Secret are needed in user service for authentication purposes,

It is needed for 2 main reasons – storing such variables in a special file makes it easier to change them in case this is needed as they are stored in place, and another reason is safety, as such .env files have some confidential data in them, like jwt-secret, we can keep them locally without storing them on GitHub by the usage of .gitgnore file.

Another important file to mention is app.yaml which is used to deploy a service to the Google App Engine. There is not much to say about the code written there, as this is just configurational data for the Google App Engine, like node.js version, entrypoint file and the default URL however, it is mentioned as this piece of code is vital for making the platform truly scalable.

```
runtime: nodejs20
entrypoint: node index.js

handlers:
  - url: /.*
    script: auto
```

Now, when configurational files are covered, the proper description of the index.js file can be made. In this part, I am mainly explaining it taking the user

service as an example, however, some functions from other services are going to be covered as well.

So, for the user service, these libraries were used:

- express: Web server framework.
- body-parser: Parses incoming request bodies.
- cors: Enables cross-origin resource sharing.
- jsonwebtoken (jwt): Manages JWT for authentication.
- bcrypt: Hashes passwords securely.
- Spanner: Google Cloud database client.
- uuidv4: Generates unique IDs (UUIDs).
- dotenv: Loads environment variables

Most of these libraries are also used in every other service, except for bcrypt as it is used to save authentication data securely.

The endpoints were created for different actions to connect the backend logic to the frontend. For example, this is how user creation looks in the backend:

```
// Registration endpoint
app.post('/api/auth/register', async (req, res) => {
  const { name, email, password } = req.body;

  try {
    const userId = uuidv4();
    const hashedPassword = await bcrypt.hash(password, 10);

    await database.runTransactionAsync(async (transaction) => {
      const query = {
        sql: `INSERT INTO users (id, name, email, password)
              VALUES (@id, @name, @email, @password)`,
        params: {
          id: userId,
          name: name,
          email: email,
          password: hashedPassword,
        },
      };
```

```
      await transaction.runUpdate(query);
      await transaction.commit();
    });

    const token = jwt.sign({ userId }, process.env.JWT_SECRET, { expiresIn: '1h' });
    res.json({ token });
  } catch (error) {
    console.error('Registration error:', error);
    res.status(400).json({ message: error.message });
  }
});
```

App.post('/API/auth/register', async (req, res)) describes what kind of HTTP request is being made and where it goes with a relative path. Variables 'req' and 'res' are used to tell which way the data goes: if it is in a response or request body. In this endpoint, the name, email and password provided by the user are sent into a request body and written into the database, with the password being hashed using the bcrypt library and the ID being generated randomly using the UUID library.

The database is connected to the service in this piece of code:

```
const spanner = new Spanner({
  projectId: process.env.SPANNER_PROJECT_ID,
});

const instance = spanner.instance(process.env.SPANNER_INSTANCE_ID);
const database = instance.database(process.env.SPANNER_DATABASE_ID);
```

In the .env file, all the necessary IDs are written, and in the backend logic, the spanner instance and database are initiated. This is the piece that appears in every single service with an adjustment of which databases are used. For example, for messaging service, two databases were used: messaging-db and user-db:

```
const spanner = new Spanner({
  projectId: process.env.SPANNER_PROJECT_ID,
});

const instance = spanner.instance(process.env.SPANNER_INSTANCE_ID);
const database = instance.database(process.env.SPANNER_DATABASE_ID);
```

```
const usersDatabase = instance.database(process.env.SPANNER_USERS_DATABASE_ID);
```

Another function that is necessary for every service is a middleware that handles JWT authentication:

```
// Middleware for JWT authentication
async function authenticateToken(req, res, next) {
  const token = req.headers['authorization'];

  if (!token) {
    return res.status(401).json({ message: 'Authorization token is required' });
  }

  try {
    const bearerToken = token.split(' ')[1];
    const decoded = jwt.verify(bearerToken, process.env.JWT_SECRET);
    req.user = decoded;

    next();
  } catch (error) {
    console.error('Token verification failed:', error);
    return res.status(403).json({ message: 'Invalid or expired token' });
  }
}
```

It is used to ensure reliable handling of authentication tokens, which are used to check who gets access to certain information, like a list of applications, conversations, or which buttons have to be shown to the user.

Another important step in this endpoint is the signing of the token, which is used across all services to ensure authorised access to different functionality of the platform. The last part of each endpoint is dedicated to handling different errors that may occur while it is active.

This endpoint is one of the simpler ones, but it is a good example of how the backend logic is structured throughout the project. With backend logic outlined, it is time to look at the frontend side of the registration process:

```
export async function register(name, email, password) {
  const response = await axios.post(`${API_URL}/api/auth/register`, { name, email, password });
  return response.data;
```

```
}
```

This function is used to send API requests to the registration endpoint. For this purpose, a library called ‘Axios’ is used to send HTTP request to the backend URL with necessary values. What happens next is described in a function:

```
const handleRegister = async (e) => {
  e.preventDefault();
  try {
    const data = await register(name, email, password);
    localStorage.setItem('token', data.token);
    navigate('/profile');
  } catch (error) {
    setError(error.response.data.message);
  }
};
```

This function is responsible for what happens after the user clicks the ‘Register’ button. If the registration is successful, he gets forwarded to the profile page, where users can check the opportunities created by them and applications to other opportunities made by them.

The next important part of the project is opportunity posting. In the opportunity service, there is a dedicated endpoint for it. As the logic behind this endpoint is quite simple, in this chapter, the only provided part is an sql query that shows which data is included in an opportunity:

```
await database.runTransactionAsync(async (transaction) => {
    const query = {
      sql: `INSERT INTO Opportunities (OpportunityID, Title, Description, Location, Deadline, PostedBy,
EducationLevel, SubjectFilters)
          VALUES (@opportunityID, @title, @description, @location, @deadline, @postedBy, @educationLevel,
@subjectFilters)`,
      params: {
        opportunityID,
        title,
        description,
        location,
        deadline,
        postedBy: request.user.userId,
        educationLevel,
```

```
      subjectFilters,
    },
  };
```

For every opportunity in the database the following data is stored – ID, title, description, location, where the work on the opportunity is planned to be conducted, application deadline, necessary level of education, subject field of the opportunity and for technical reasons also a user ID of opportunity creator is stored.

On the frontend side, every field, that is filled by the user, is structured like this:

```
<div>
  <label>Title</label>
  <input
    type="text"
    value={title}
    onChange={(e) => setTitle(e.target.value)}
    required
  />
</div>
```

Before sending all the input data to the backend the React hooks are created for each type of value. For this example, it looks like this:

```
const [title, setTitle] = useState('');
```

The React hook ‘useState’ initialises with an empty string and then is filled with data that was input in the text field and then sent in POST request to the backend.

Another important functionality realised in the opportunity service is application handling. The decision to put this functionality into the opportunity service was made because applications are dependent on the opportunities, so it made sense to combine them in one service. An important part of applying is attaching a CV, which is sent to the server as a file. And this process is handled by the following function:

```
if (req.file) {
  try {
```

```
    const blob = bucket.file(`cvs/${applicationID}-${req.file.originalname}`);
    const blobStream = blob.createWriteStream({
      resumable: false,
      metadata: {
        contentType: req.file.mimetype,
      },
    });

    blobStream.on('error', (err) => {
      console.error('Error uploading file to GCS:', err);
      res.status(500).json({ message: 'Failed to upload CV' });
    });

    blobStream.on('finish', () => {
      cvUrl = `https://storage.cloud.google.com/${bucket.name}/${blob.name}`;
      console.log('File uploaded to GCS:', cvUrl);
      submitApplication();
    });

    blobStream.end(req.file.buffer); // Uploading the file buffer to GCS
  } catch (err) {
    console.error('Error handling file upload:', err);
    return res.status(500).json({ message: 'Failed to process CV upload' });
  }
}
```

This code snippet, as said before, handles the upload of the CV file to Google Cloud Storage (GCS), and to do so, there are several stages. Firstly, a check is made to ensure that the file is provided at all (req. file). After that, a file is created in the GCS bucket. A reference to the file in the GCS bucket is created using the unique applicationID and the original file name (req.file.originalname). The next step is opening a write stream (blob.createWriteStream) to upload the file to GCS bucket. The type of the uploaded file is included in the metadata for proper handling of the file type.

If there's an error during the upload process, the function logs the error and sends a 500 status response back to the client, indicating that the file upload failed.

If the upload is finished, the URL of the uploaded file in GCS (cvUrl) is generated and logged. Then, the submitApplication() function is called to

continue the process of submitting the application, and the link to the CV is saved to the database with other data.

For this to work, GCS bucket should be initiated in the service. It was done this way:

```
const storage = new Storage();
const bucket = storage.bucket(process.env.BUCKET_NAME);

const upload = multer({ //creating file storage
  storage: multer.memoryStorage(),
  limits: { fileSize: 10 * 1024 * 1024 },
});
```

In this code snippet, storage is initiated, and after that, file storage for each CV is created using the 'multer' library, included in the opportunity service.

However, from the frontend side, this process is structured similarly to creating an opportunity. The only important addition is a special object that handles files:

```
  const formData = new FormData();
  formData.append('motivationLetter', motivationLetter);
  if (cvFile) {
    formData.append('cv', cvFile);
  }
```

This code creates a FormData object to prepare data for an HTTP request. This object stores the CV file if it is uploaded, and it also stores the motivation letter, another part of the application, which is a text input. After that, the object submits this data in one request.

With the most interesting parts of opportunity service covered, I am now going to cover some highlights from search service and messaging service, starting with a search service.

Taking into consideration that the search service mostly consists of some filtering functionality, these are the filters that are used for the project:

```
//Filters:
if (title) {
  sql += ` AND LOWER(Title) LIKE LOWER(@title)`;
  params.title = `%${title}%`;
}
if (location) {
  sql += ` AND LOWER(Location) LIKE LOWER(@location)`;
  params.location = `%${location}%`;
}
if (educationLevel) {
  sql += ` AND EducationLevel = @educationLevel`;
  params.educationLevel = educationLevel;
}
if (subjectFilters) {
  const subjectsArray = subjectFilters.split(',').map(s => s.trim());
  sql += ` AND EXISTS (SELECT subject FROM UNNEST(SubjectFilters) AS subject WHERE subject IN
UNNEST(@subjectFilters))`;
  params.subjectFilters = subjectsArray;
}
```

This snippet allows filtering opportunities by four variables: title, location, education level, and subject. To ensure that no opportunity is missed, the registers are ignored with the 'LOWER' parameter, so if a user searches for 'London' as the location, they will get the result even if, in an opportunity, the location is marked as 'London'.

Some of the functions to mention are in the messaging service. This service manages the creation of new conversations and messages as well as sending email notifications to the recipient about new messages. While the creation of a conversation is quite a simple process, it is important to cover the message-sending endpoint on the backend and its frontend part. The message-sending endpoint has a few interesting functions:

```
async function sendEmailNotification(recipientEmail, subject, messageText) {
  try {
    let transporter = nodemailer.createTransport({
      host: process.env.SMTP_HOST,
      port: process.env.SMTP_PORT,
      secure: true,
```

```
    auth: {
      user: process.env.SMTP_USER,
      pass: process.env.SMTP_PASS,
    },
  });
  let info = await transporter.sendMail({
    from: `"Messaging App" <${process.env.SMTP_USER}>`,
    to: recipientEmail,
    subject: subject,
    text: messageText,
  });

  console.log('Email sent: %s', info.messageId);
 } catch (error) {
  console.error('Error sending email:', error);
 }
}
```

Using the 'nodemailer' library, the SMTP transporter is created using the credentials provided in the .env file for this service. This function sends an email to the recipient's address, which is fetched in the next snippet and logs the result of the operation, whether it is successful or not.

```
const [userRows] = await usersDatabase.run({
      sql: `SELECT email FROM users WHERE id = @recipientID`,
      params: { recipientID },
    });

    console.log('Result from users table:', userRows);

    const recipientEmail = userRows.length > 0 && userRows[0][0].value ? userRows[0][0].value : null;

    if (!recipientEmail) {
      console.log('No email found for recipient:', recipientID);
      return res.status(400).json({ message: `Recipient email not found for user: ${recipientID}` });
    }

    const subject = 'You received a new message';
    const emailText = `You received a new message from user ${senderID}: ${messageContent}`;
    await sendEmailNotification(recipientEmail, subject, emailText);
```

This snippet is located inside the endpoint that handles the sending of the message, and it is used to fetch the user's email from the user database, comparing the ID from the conversations database to the ID located in the user database. If this is successful, the function described earlier is triggered to send

a notification to email alongside the message itself on the platform. The notification has the “You received a new message” text with the message content and sender’s ID.

From the front end, it may be interesting to look at how sending a request to the back end is handled.

```
const handleSendMessage = async () => {
   const token = localStorage.getItem('token');
   if (!newMessage.trim()) return;

   try {
     await
axios.post(`${process.env.REACT_APP_BACKEND_MESSAGING_URL}/api/conversations/${conversationID}/m
essages`, {
       senderID: jwtDecode(token).userId,
       messageContent: newMessage,
     }, {
       headers: {
         Authorization: `Bearer ${token}`,
       },
     });
     setNewMessage('');
     window.location.reload();
   } catch (error) {
     setError('Failed to send message');
   }
 };
```

For everything to work correctly after sending the POST request using ‘axios’ with message content and sender ID, it was decided to reload the window. Hence, a new message appears on the screen instantly.

While this chapter of the project report shows some functions and important code snippets, it is impossible to cover them all here. However, the presented materials with descriptions should provide insight into how the development was conducted, which instruments and methods were used, and how some details of the functionality were achieved. The full source code is located in Appendix B and on the GitHub repository: https://github.com/DUVIED101/MScProject.

Chapter 4 - Results

4.1 Development result

As a result of the development process, the developed platform consists of 4 backend microservices with one frontend application, all running in Google App Engine and available via the link: https://frontend-dot-custom-mile-430418-b9.uc.r.appspot.com/. Some errors that may appear on the screen may not necessarily be errors on the side of the platform but some business logic. For example, if there are no conversations for the user, he may see a "failed to fetch conversations" error. It is important to consider this when testing the platform. Unfortunately, the platform is going to be available via this link only up until the 24th of October, with some features stopping to work even earlier. After that date, most of the functionality may not be available due to the end of the trial period on Google Cloud. However, the product is also accessible through Github and can be redeployed.

I've also prepared a a short demonstration video, showing the functionality of the application it can be found here: https://drive.google.com/file/d/1wKLu2AbZhWxqbOWFeMvoyLKkJSB7vpwV/view?usp=sharing

This part is written to demonstrate which functionality was implemented with screenshots and other evidence. It is important to mention that due to the limited amount of time given to the development process and Google Cloud's trial restrictions, some functionality may work with a few problems, for example, some UI elements may be shown incorrectly.

Firstly, it is important to show how the authentication was implemented. 2 pages were created for that matter: Registration and Login pages.

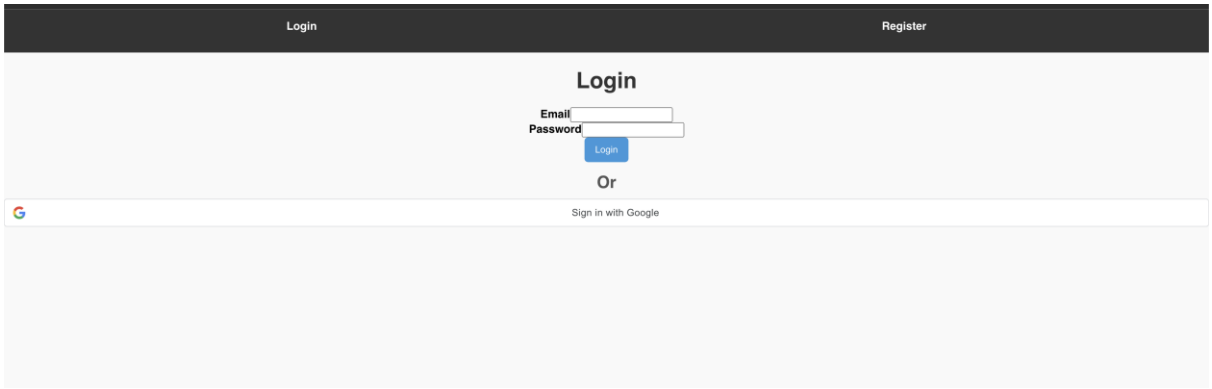


Figure 5 - Login Page

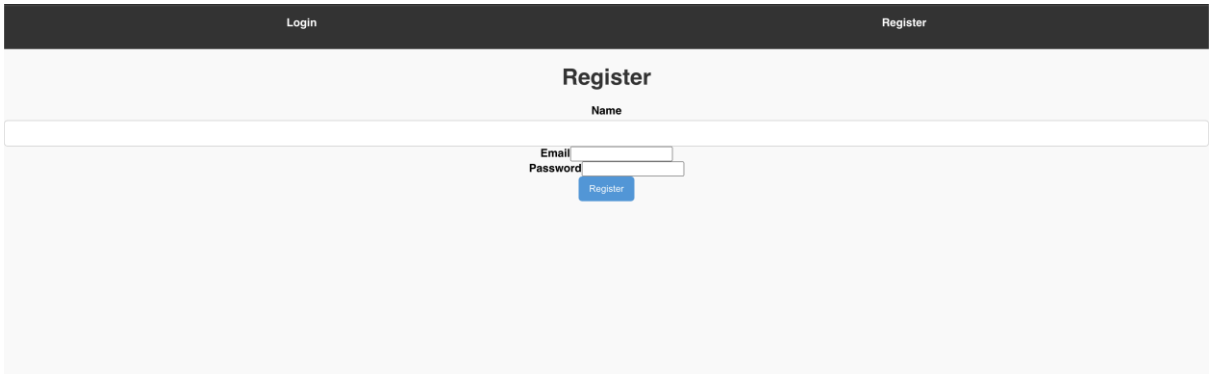


Figure 6 - Registration page

These pages provide simple authentication functionality. When registering, users input their name, it is used in messaging, for example.

After the authentication process users are redirected to the profile page, where important profile information is shown.

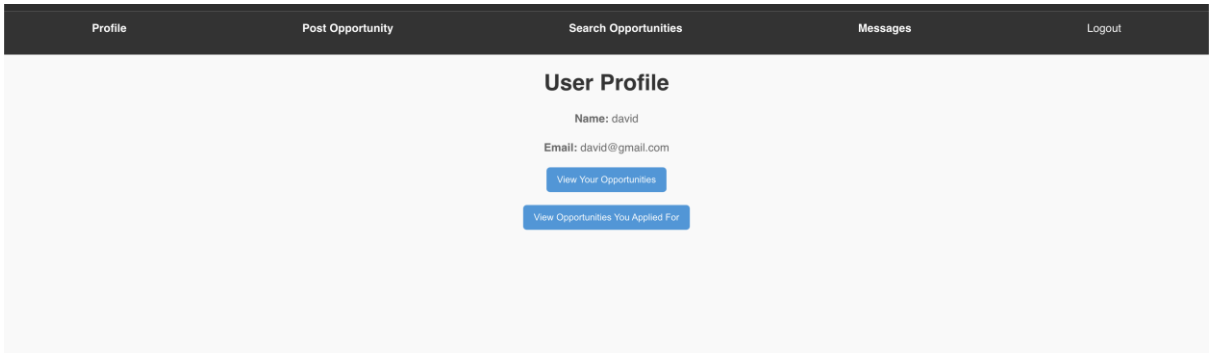


Figure 7-Profile page

This page also has a different navigation bar, that leads to core platform capabilities, creating and searching opportunities and messenger. Another two

buttons on the lower part of Figure 7 are opportunities created by the user and opportunities the user applied for.

If the user clicks goes the page dedicated to opportunity posting, this is what is displayed.

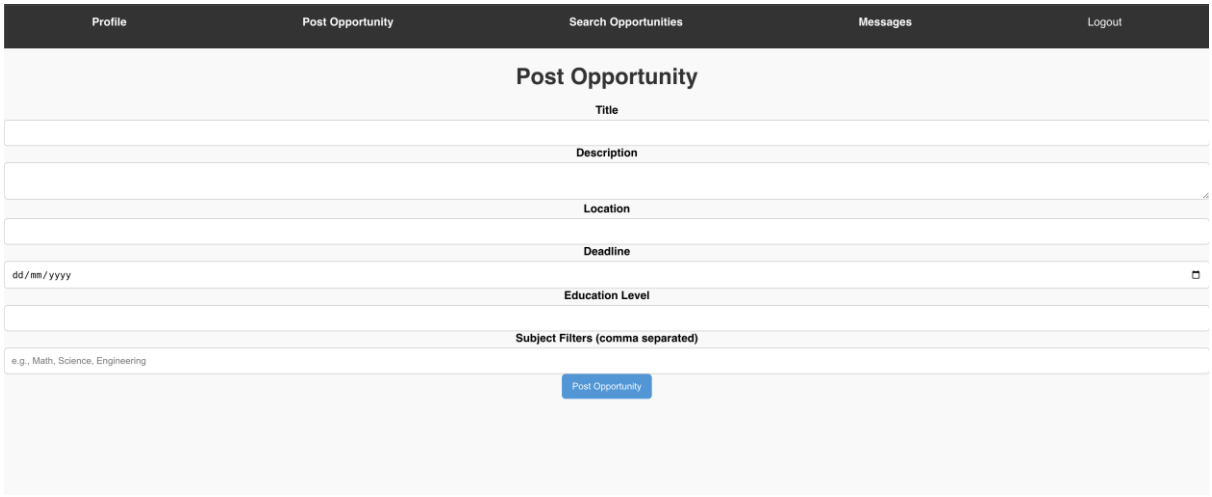


Figure 8 - Opportunity Posting Page

Created opportunities can be seen on the search page:

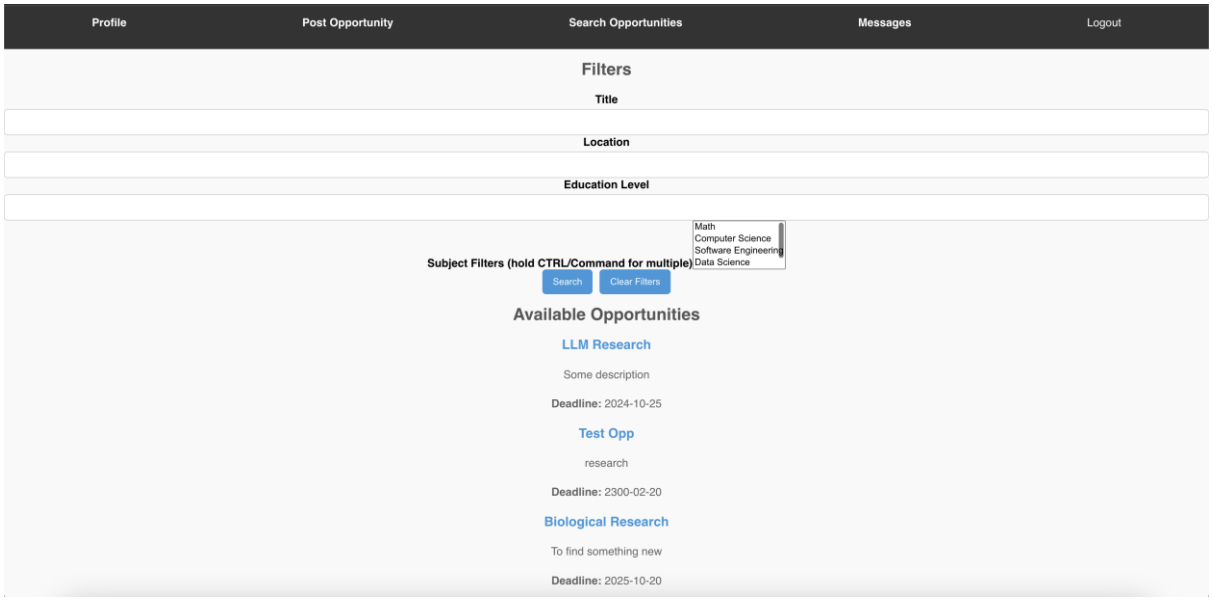


Figure 9 - Search Page

By default, all opportunities are shown, however, if the user wants to filter them, it is done in the upper part of the page.

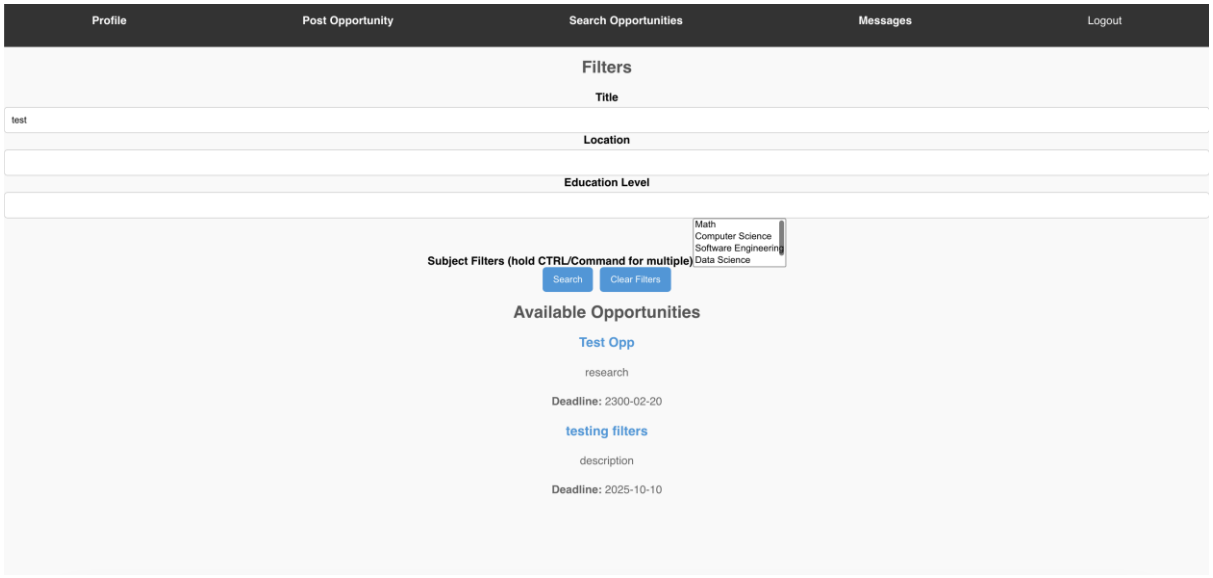


Figure 10 - Search Page (with filters)

Opportunity titles are click responsive and lead to the opportunity info page. If specific opportunity was created by the current user, he is going to see a button “View Applications for this Opportunity”. Otherwise, the button “Apply for this opportunity” is shown.

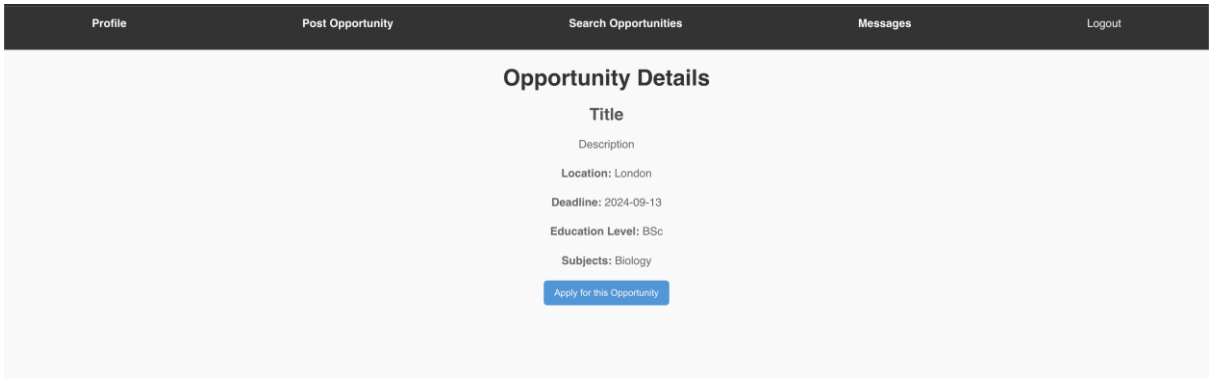


Figure 11 - Opportunity Information Page

Applying for the opportunity is a simple process, users only need to provide a motivation letter and their CV.

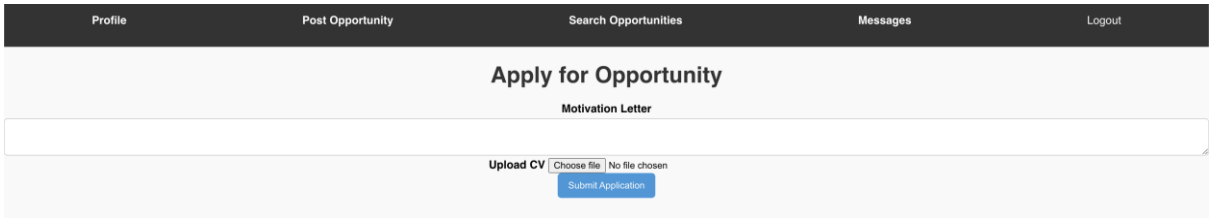


Figure 12 - Application Page

Applications, made by users can be seen from the profile page.

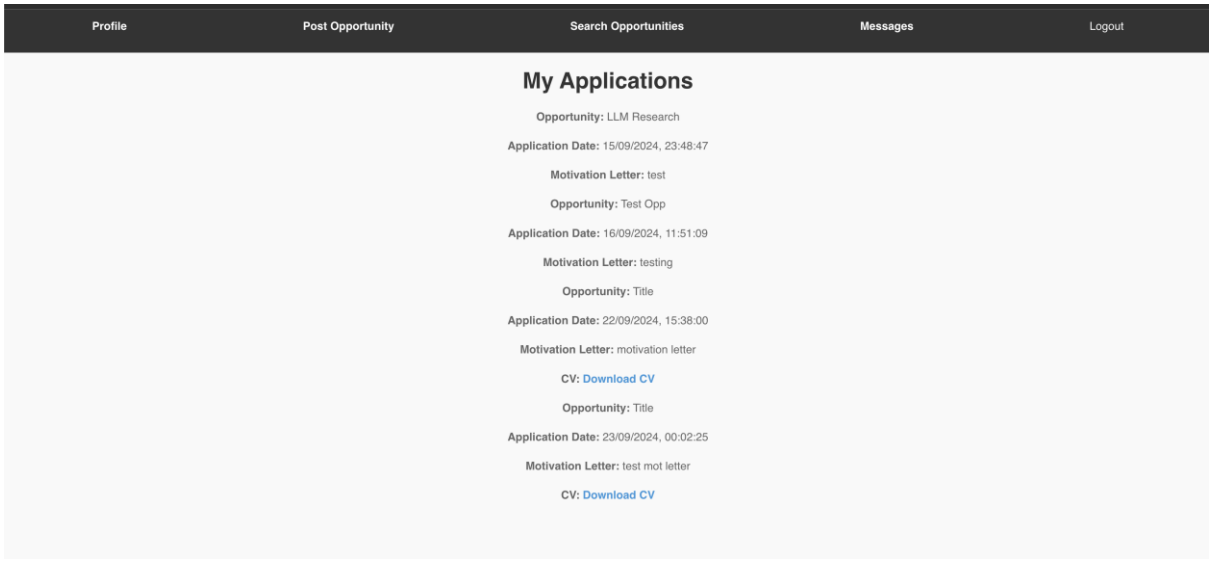


Figure 13 - User's Applications Page

On the other hand, the creator of the opportunity can access all applications that were made to his opportunity from the opportunity info page. All opportunities created by specific users are also accessed from the profile page.

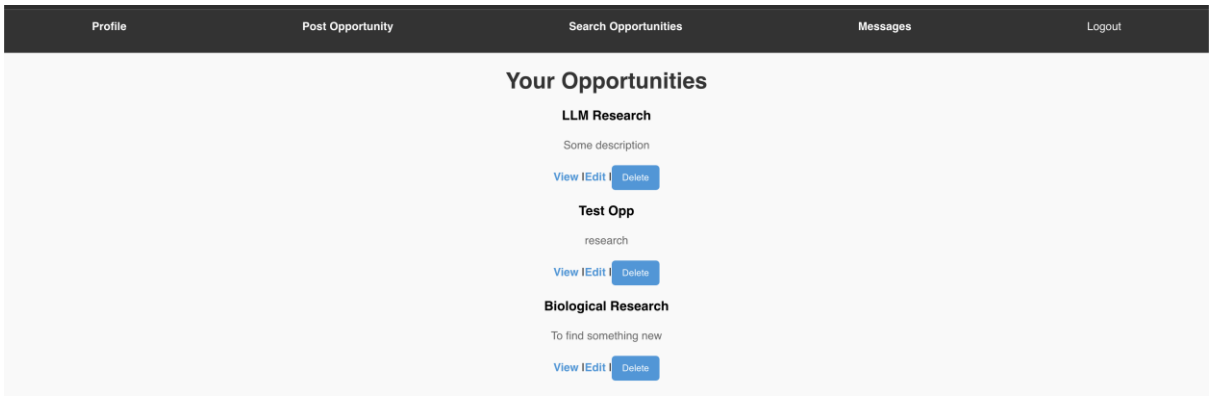


Figure 14 - User's Opportuinities Page

Clicking “view” redirects to the opportunity information page, where the button to show all the applications appears.

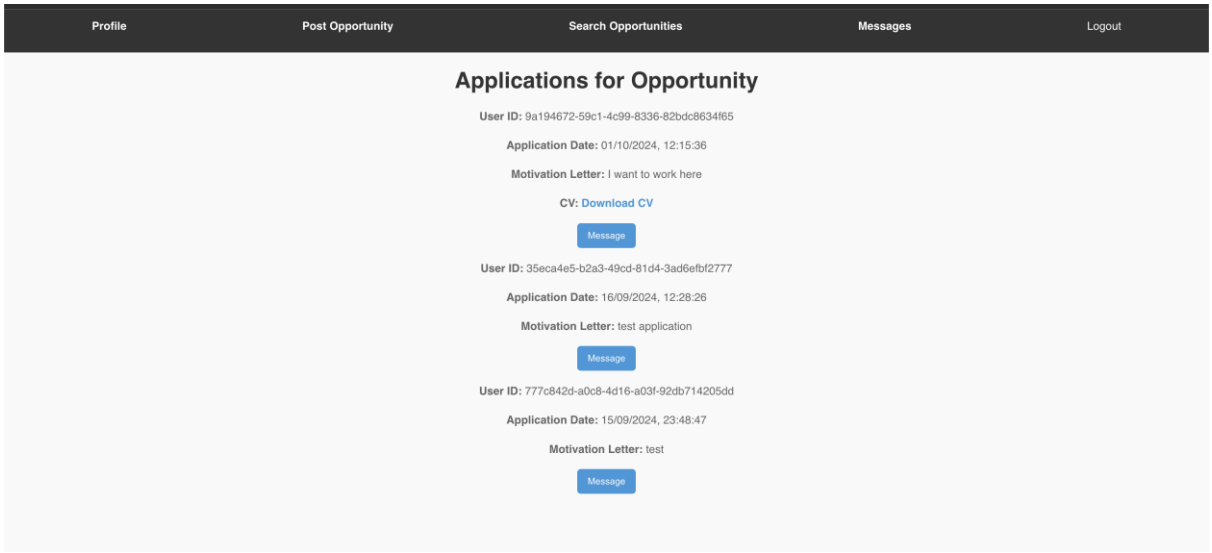


Figure 15 - Applications for Opportunity Page

This page has a Message button, clicking on which creates a new conversation that can be found on the Messages page.

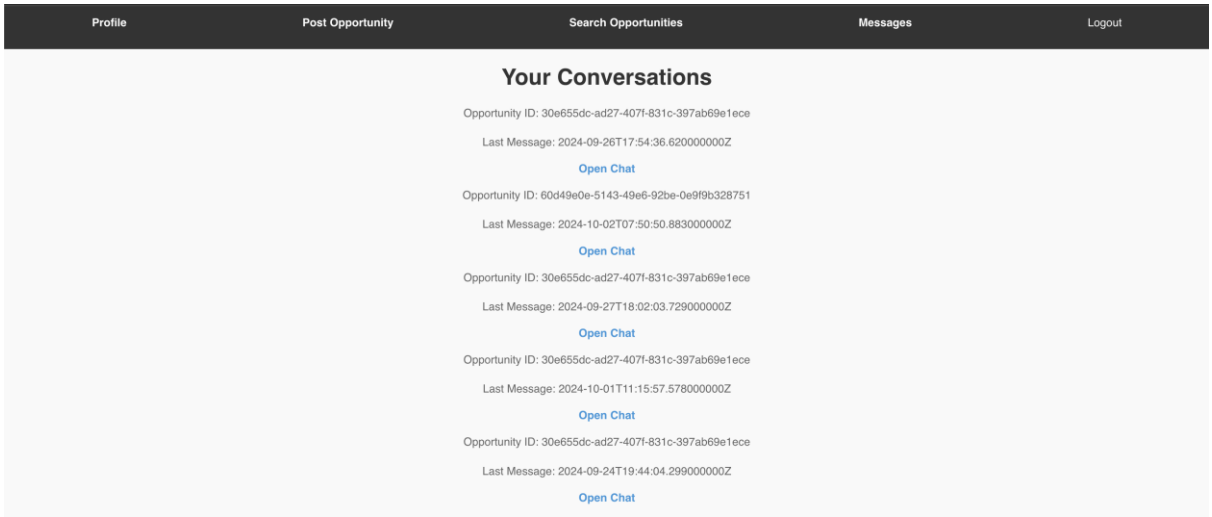


Figure 16 - Conversations Page

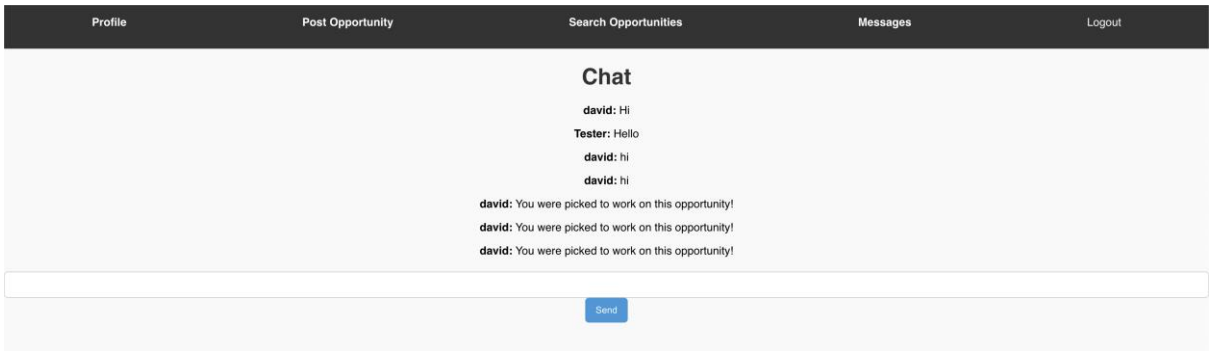


Figure 17 - Chat Page

The last important functional detail is message notifications. When a message is sent, the recipient receives an email telling what is the message's content.

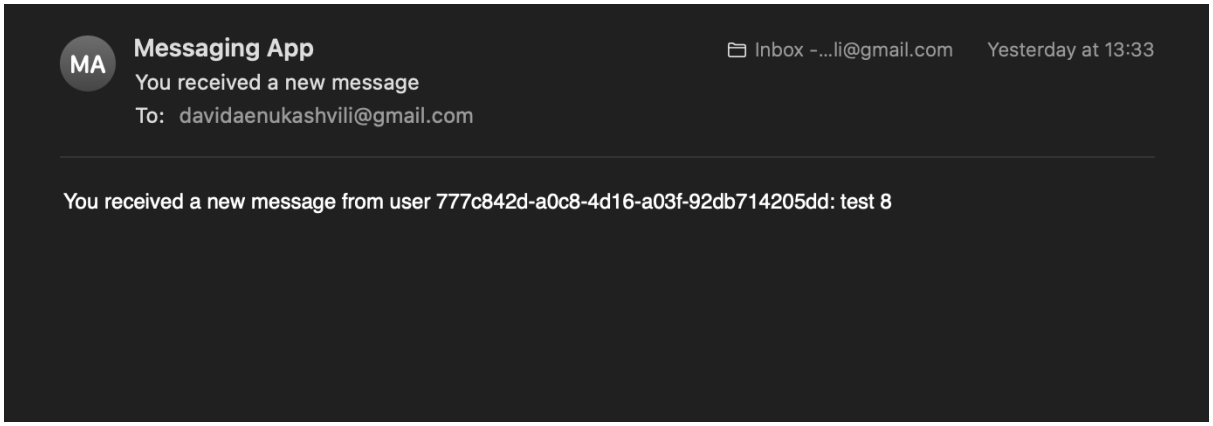


Figure 18 - Email Notification

Figures 5-18 show the core functionalities implemented into the platform. This is how the platform is operated from the user side. Now it is important to cover, how the cloud infrastructure works with all services deployed.

All important information – current load, number of requests to the services and average latency is shown on a Dashboard page of Google App Engine.

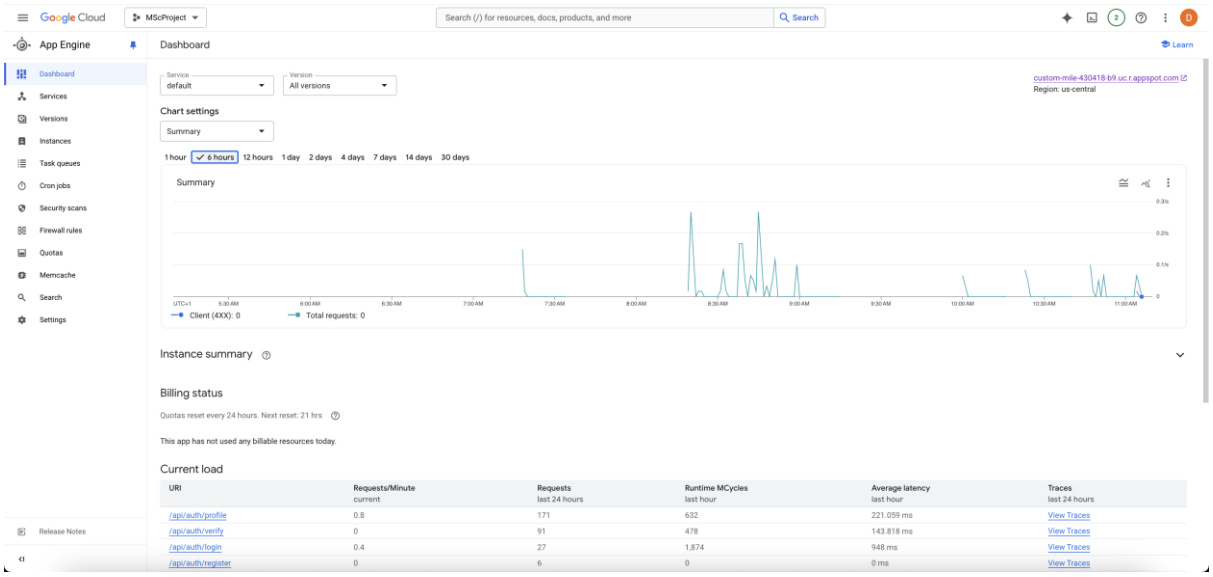


Figure 19 - Google App Engine Dashboard Page

This covers the most important results of the development– implemented functionality and how it works when deployed on a cloud infrastructure.

4.2 Testing

Unfortunately, not all tests planned for this project were conducted; some unit tests were done, and their results are provided in the following figures. While an insufficient amount of tests were created, this does not mean that project objectives are not done as all functionality required to declare the project as ready is present on the website, provided at the beginning of chapter 4 and evidence of that is also provided in figures 5-18.

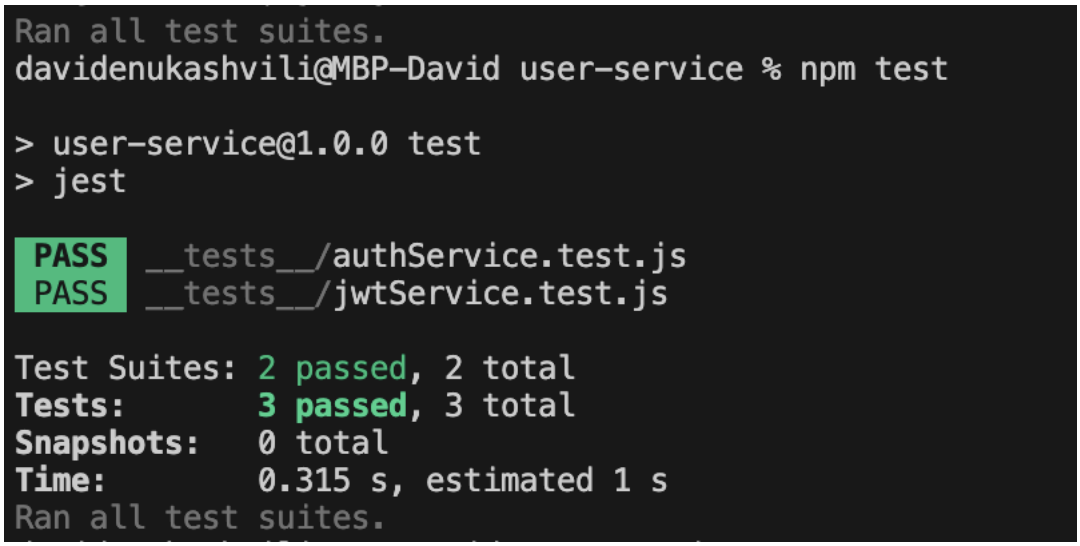


Figure 20 - Example of the passed Unit Tests

Another important thing to mention is that, for example, there is no need to conduct specific scalability tests. This would be important to do if the infrastructure was deployed manually on the Kubernetes engine. Taking into consideration that the project is deployed on the Google App engine, which is managed automatically, it should be scalable by definition.

Chapter 5 - Evaluation, Reflections, and Conclusions

5.1 Results Evaluation and Reflection

5.1.1 Comparing results to objectives

As said before, three key objectives had to be achieved – Design a Scalable Application, Enable Sharing of Multidisciplinary Opportunities and Facilitate Collaboration. Taking the outcomes of the project into consideration, all three of these objectives can be declared as achieved.

The application is scalable thanks to the usage of the Google App Engine for the deployment – the service manages all services automatically and is going to scale them according to the current load which is exactly what needed to be done.

Sharing of Multidisciplinary Opportunities is achieved, as there is a possibility to post different opportunities in different scientific fields, supported by the presence of a search engine that helps to filter opportunities by different categories – from titles to education levels and scientific fields. Every user, registered on the platform can access and apply for the opportunities, so the objective can be declared as met.

Collaboration on the platform is possible due to the presence of applying functionality, which allows opportunity creators to find candidates for the opportunity and the presence of a messaging system, which allows them to discuss any questions and details before proceeding with actual work on the opportunity. Thus, the objective of facilitating collaboration can be declared as met.

5.1.2 Problems encountered

Despite the fact all objectives are met, it is important to discuss what has not been achieved during the work on the project. While all basic functionality is present, and the platform can operate in its current state, some features were not fully developed and are present as prototypes of possible functionality. This is especially true for two components – the notification system and the search engine.

Initially, it was planned to create a specific notification service that would handle different types of notifications. For example, notifications of a new application made to the opportunity. It was also planned to create not only email notifications, which was a key feature but also internal notifications that would be sent directly to the front-end side of the app, maybe in a special section. As a result, while notifications are present in the platform as it was promised in the project proposal, they became part of the messaging service, because they are only used to send notifications about new messages.

For the search engine, there was an initial plan to create a comprehensive list of filters, especially for subject topics. One of the ways to filter opportunities that can be categorised as computer science was to use the ACM Computing Classification System (Association for Computing Machinery, 2024). However, the current version of searching engine only provides a few basic filters, which is enough to provide the required functionality, but it was planned to do extra work in the field.

Another major problem was not directly in the development process; it was a problem with testing. I dedicated a few sentences to this at the end of the result section, but it is important to mention that not enough testing was carried out. Some parts of testing could be missed due to the infrastructure specifics: like was said before, scalability testing was not necessary due to the

fact the platform is scalable by definition because it uses Google App Engine. However, not enough unit and integration tests were created for the platform. It may not be critical for the project it is a form of studying prototype, but in case it gets to the production stage, testing has to be done in another, more complete form.

5.1.3 Final Reflection

This project was challenging and interesting to work on. It helped me to learn new technologies – I was not familiar with JavaScript and its libraries before the work on this project, as well as dive deeper into technologies I was familiar with before, like the Google Cloud platform.

Working on this project also highlighted the importance of time management and thoroughly planning the work because it made me face the issue of not having enough time for some problem-solving, as was discussed previously.

If I had to do this project from the beginning with the knowledge I gained, I would plan my time better to have some spare time for problem-solving and also searching for more appropriate literature as it was hard to find, taking into consideration there is not much work in this field. Another challenge that was not taken into consideration is the fact that the whole work was to be done in English. As it was the first time I worked on a project of this scale in this language, it would be wise to give myself more time to gather and structure the information for this report.

5.2 Conclusion

5.2.1 Project conclusion

This project achieved its main objectives and is a scalable opportunities platform that can be used to post opportunities and find the best-suiting candidates to work on them. However, it is not ideal, and there are a lot of

things that can make the project better. Despite this, the work conducted on the Academic Opportunities Platform shows that it is possible to create a centralised hub that will establish communication in the academic world which is similar to the one present in the corporate world.

5.2.2 Possible further work

Further work may be undertaken to tackle the issues described in chapter 5.1.2. Solving them will make this platform ready for production use. Another possible upgrade that can be made in the future, is the addition of the recommendation system that can make searching for an opportunity even more efficient and easy process.

References

DUVIED101, 2024. MSc Project Repository. GitHub. [online] Available at: https://github.com/DUVIED101/MScProject [Accessed 02 Oct. 2024].

MSc Project, 2024. Academic Opportunities Platform. [online] Available at: https://frontend-dot-custom-mile-430418-b9.uc.r.appspot.com/ [Accessed 02 Oct. 2024].

Bondar, O., 2023. Important LinkedIn Statistics, Data & Trends. LinkedIn Pulse, [online] Available at: https://www.linkedin.com/pulse/important-linkedin-statistics-data-trends-oleksii-bondar-pqlie/ [Accessed 14 May 2024].

LinkedIn, 2024. About Us. LinkedIn Statistics, [online] Available at: https://news.linkedin.com/about-us [Accessed 14 May 2024].

Indeed, 2024. About Indeed. Indeed Statistics, [online] Available at: https://uk.indeed.com/about [Accessed 14 May 2024].

Ofcom, 2022. Adults' Media Use and Attitudes 2022: Interactive Report. Ofcom Report, [online] Available at: https://www.ofcom.org.uk/__data/assets/pdf_file/0028/241275/Adults-Media-Use-and-Attitudes-report-2022.pdf [Accessed 14 May 2024].

Wuchty, S., Jones, B.F. and Uzzi, B., 2007. The Increasing Dominance of Teams in Production of Knowledge. Science, 316(5827), pp.1036-1039. Available at: https://typeset.io/papers/the-increasing-dominance-of-teams-in-production-of-knowledge-h591ohwfzk [Accessed 5 June 2024].

Xiao, X. and Prompanyo, M., 2021. Influence of Scientific Collaboration Network on Academic Performance. The International Journal of Business and Management, 16(12), pp.68-. Available at: https://doi.org/10.5539/IJBM.V16N12P68 [Accessed 6 June 2024].

Business of Apps, 2024. LinkedIn Statistics. Business of Apps, [online] Available at: https://www.businessofapps.com/data/linkedin-statistics/ [Accessed 15 May 2024].

Boden, R. and Epstein, D., 2006. Opportunities in the Post-Academic World. Studies in Higher Education, 31(5), pp.569-585. Available at: https://typeset.io/papers/opportunities-in-the-post-academic-world-3w6m4w6j7l [Accessed 2 July 2024].

International Labour Organization, 2020. World Employment and Social Outlook: Trends 2020. ILO Report, [online] Available at: https://typeset.io/papers/world-employment-and-social-outlook-trends-2020-lmuc4uj2sk [Accessed 1 July 2024].

Oxford IT Services, 2024. Research Collaboration Tools. [online] Available at: https://www.it.ox.ac.uk/research-collaboration-tools [Accessed 29 July 2024].

Harvard Data Management, 2024. Collaborative Tools & Software. [online] Available at: https://datamanagement.hms.harvard.edu/collect-analyze/collaborative-tools-software [Accessed 7 Aug 2024].

Konfer Online, 2024. [online] Available at: https://konfer.online/ [Accessed 1 May 2024].

Academic Positions, 2024. [online] Available at: https://academicpositions.com/ [Accessed 12 July 2024].

Bui, N., Anggorojati, B., Jang, J. and Kim, K., 2021. Security and Privacy in Cloud Computing: Data Protection and GDPR. IEEE Access, 9, pp.104-117. Available at: https://ieeexplore.ieee.org/document/9000678 [Accessed 14 Sep 2024].

Chen, J., Zhang, Z., Wang, Y., and Li, W., 2019. Dynamic Cloud Resource Allocation: Balancing Performance and Costs. Journal of Cloud Computing, 8(3), pp.45-60. Available at: https://journalofcloudcomputing.springeropen.com/articles/10.1186/s13677-019-0132-x [Accessed 14 Sep 2024].

Fernández, A., García, S., Luengo, J., and Herrera, F., 2018. Cloud Services for Academia: Benefits, Challenges, and Recommendations. Journal of Computing in Higher Education, 30(1), pp.1-15. Available at: https://link.springer.com/article/10.1007/s12528-017-9165-9 [Accessed 14 Sep 2024].

Google Cloud, 2024. Security in Google Cloud. [online] Available at: https://cloud.google.com/security [Accessed 14 Sep 2024].

Zhao, Y. and Liu, L., 2020. The Role of AI and Machine Learning in Cloud Computing and Future Digital Services. IEEE Transactions on Cloud Computing, 8(4), pp.120-133. Available at: https://ieeexplore.ieee.org/document/9012345 [Accessed 14 Sep 2024].

Association for Computing Machinery (ACM), 2024. ACM Computing Classification System. [online] Available at: https://dl.acm.org/ccs [Accessed 25 Sep. 2024].

Appendix A – Project Proposal

Introduction

The purpose of the project is to design and develop a scalable platform that would enable the members of the academic community (professors, students, universities, etc.) to create and find academic opportunities of different profiles.

In the last decades, the way people find a job changed drastically to using platforms like LinkedIn and Indeed to connect with people and find vacancies through them or direct job postings. My idea is that a similar approach can be used to help the academic community post their opportunities and to other members to work on them.

This platform could facilitate access to diverse opportunities: research programs, internships, and other sorts that the academic world is full of, making it more accessible. A scalable academic opportunities platform would require the development of a system that can meet the needs of students and other members of the academic community across disciplines.

Project Objectives:

1. Design a Scalable Platform: Design a flexible and scalable platform with the use of Cloud Technology to create a platform that will be able to perform consistently under different levels of load.
1. Enable Sharing of Multidisciplinary Academic Opportunities: It does include creating a platform for sharing many academic opportunities, starting from research projects to experimental activities for publication co-authorship and other types of opportunities.

2. Facilitate Collaboration: Make mechanisms to facilitate the academic collaboration of different members of an academic community that wouldn't find each other without such a platform.

Critical Context

One of the main inspirations for this project is the popularity of platforms for searching for jobs, particularly LinkedIn and Indeed. In the last decade, platforms like this grew massively [1] and [7] as searching for a job online became a more common thing. Numbers speak for themselves: LinkedIn has more than a billion users across the world [2] and more than 300 million use it every month [4] and Indeed is visited by 350 million users every month [3]. Overall, according to the survey [4], 37% of adults in the UK used internet platforms to find a job, and around 50 per cent if we look at the age gap of 16-44.

Another interesting thing with platforms like Indeed is the availability of global job search. Finding the most qualified employee is now possible even if they live on another continent. It works the same way for the candidates, it is possible to find an ideal employer for candidates' skills and ambitions. It wasn't like this in the pre-internet era and it is a reality now.

Inspired by the success of these platforms, this project aims to create a similar tool focused on academic opportunities. The envisioned platform will allow academic opportunities to reach a broader audience, including students and researchers who may not have access to these opportunities through traditional means. On the other side, it will provide institutions and professors with an instrument that will assist them in finding people, interested in conducting research or participating in experimental activities. This is particularly relevant in the academic world, where collaboration and access to

diverse research opportunities can significantly enhance scholarly work and professional development [5].

The promotion of collaboration is one of the objectives of this project. From the papers [5] and [6] we can understand that establishing new connections, e.g. finding a research partner, promotes scientific performance. This can work in several ways: having someone to assist with research enhances performance by itself, but there is another benefit: after being connected via an academic opportunity platform and doing some valuable job, an established research team can come up with new ideas, further strengthening their academic impact.

Another vital part of my project is making it scalable. The platform's scalability is crucial for handling different load levels while staying efficient on allocated resources. As the platform grows, it must handle varying load levels without compromising performance. This necessitates the use of cloud technology, which offers the flexibility and scalability required to support a large number of simultaneous users and ensure consistent performance. I will further specify how it will be done, but overall, cloud technology will help me develop a platform that will provide reliable service to its users.

In conclusion, the decision to create this platform is inspired by the need to modernise and expand access to academic opportunities, leveraging the proven model of job search platforms. By utilising advanced cloud technologies and focusing on scalability and user experience, the platform aims to significantly enhance the way academic opportunities are shared and accessed globally.

Approaches: Methods & Tools for Design, Analysis & Evaluation

In this section, I outline key components of the development approach. Some details may change at the beginning of actual development and this part shows my initial thinking.

Literature Review

As part of writing a Project Proposal, I reviewed several research papers and other sources with valuable information suitable for my project. However, there will be a broader review of other papers related to the topic, as the need for this arises throughout the development process

Analysis

At the beginning of the development, before actually writing code, I am going to do thorough design and analysis. This includes creating diagrams to fully explain how the platform is planned to operate on different levels of abstraction. Another thing to do is to analyse how finding research is structured now: understanding, how academic opportunities are described to a potential performer, which codifications are used and other aspects of this field should be analysed to develop a platform that is easy to use and reaches all the goals and objectives I have.

Architecture

The architecture of the academic opportunities platform is structured using a microservices approach, designed to ensure scalability, flexibility, and resilience. This architecture divides the platform into several smaller,

independent services, each responsible for a specific functionality. Here is a list of specific services that are going to be made

User Management Service: Manages all user account functionalities such as registration, login, profile updating, and authentication. This service ensures secure and effective management of user data.

Opportunity Posting Service: It is a service that enables the adding, updating, and deleting of academic opportunities posted by Professors and Institutions. In this manner, the whole life cycle of the posting is managed by one service, therefore ensuring constantly actualised and updated data.

Application Management Service: All student applications made towards the various opportunities are humanly handled by an application manager who guarantees continuous tracking of each life cycle of the application right from submission to the final decision and everything else therein.

Search and Recommendation Service. Powerful search filters for users to find opportunities within their field of study, required degree level, location, and other filters, if they are applicable. They will also display recommendations for opportunities according to recommendation algorithms based on user profiling and historical interaction data.

Messaging Service: This is the communication service within the platform between users, allowing students to get in direct touch with opportunity providers.

Notification Service: This is intended to notify users of new opportunity postings that relate to their applications and other important alerts. The service guarantees users are well in touch with the system about all activities relevant to them.

Technological Stack and Methodology

Design Methodology

The scalable academic opportunities platform will be designed iteratively, because of the limited resources. This will allow us to start with a basic necessary functionality in the first iteration with the addition of extra functionality along with an analysis of done work in the next iterations. With this approach, I will be able to address challenges and concerns that may arise during the development cycle individually.

Technical Stack

Cloud Infrastructure: Google Cloud Platform (GCP)

Selection Rationale:

The decision to use Google Cloud Platform (GCP) is made based on its scalability, reliability, and wide range of instruments that support all phases of cloud-based application development. GCP provides tools for computing, storage, networking, and security, all crucial for the development of a scalable academic opportunities platform. Another important advantage is integration with the Google ecosystem, e.g. Google Identity Platform.

Comparison with Alternatives:

Amazon Web Services (AWS): AWS is another cloud service provider with similar functionalities. GCP is chosen because of its integration with the Google ecosystem, including Google Workspace and Google Identity Platform. These instruments may be used in the development process.

Microsoft Azure: Azure offers strong integration with Microsoft products. However, GCP is preferred for its capabilities in containerisation through Kubernetes, which is important as the project is planned to be scalable.

Frontend: React

Selection Rationale:

The reason why I went for React as my front-end framework is because of the component-based architecture it provides; it can help develop large UI components in a modular way, which bodes well for an application developed using the microservices approach. Another reason to pick React is it has a huge ecosystem, and many libraries that are already built are in place.

Alternatives:

Vue.js: In Vue.js, too, it is possible to develop interactive user interfaces. However, it is a much easier instrument compared to React. The choice goes in favour of React because of the big community around it and the modular approach which gives more support and resources to developers.

Angular: Angular is more of a full-blown framework, with more features out of the box compared to React. But the learning curve and heavyweight of it make React always a better choice.

Backend Development: Node.js

Explanation Rationale:

At the backend, Node.js is picked for its non-blocking, event-driven characteristics, which are suitable for platforms with many concurrent connections without causing a performance bottleneck. Besides, since both the application's frontend and backend have to be written using the same language, which is based on JavaScript, it will be easier to work with.

Alternatives:

Django (Python): Offers a high-level framework with a rapid development capability but does not perform as well as Node.js in handling real-time data or high concurrency.

Database: Spanner

Selection Rationale:

Google Cloud Spanner was chosen for the need to support a fully managed, distributed, and highly consistent database service for the academic opportunities platform.

Spanner combines the best parts of both traditional relational databases and typical NoSQL system scalability. It will therefore have the ability to execute complicated transactions and SQL queries holding the data relationships within the platform.

Alternatives:

Google Bigtable: Bigtable is most useful for purposes of low-latency read/write operations on huge data sets and time series data. Unlike Spanner, it does not support SQL or transactions. This fact places Spanner in a position more

appropriate for applications that have features provided by a relational database and for complex transactions across a globally distributed data set.

Google Firestore: Firestore provides a flexible, scalable database for mobile, web, and server development. It offers real-time synchronisation and offline support which are crucial for mobile applications. Spanner, in contrast, is optimised for large-scale applications that require robust transactional consistency and SQL support, which makes it better suited for the academic opportunities platform with complex query needs.

Google MySQL/PostgreSQL (Cloud SQL): These are managed versions of the traditional databases. However, they lack the horizontal scaling and global distribution capabilities, which are vital for a platform expected to handle high loads and provide consistent global access.

Testing

After the development process is ended, I will execute tests to verify that the platform can be used in planned scenarios. Extensive testing of the academic opportunities platform will consist of a few parts. Unit testing will check for correctness in each component. Integration testing will ensure the different services work together seamlessly. Automated tests for continuous integration and deployment will identify issues. A load test will examine the scalability of a platform and its functionality under any user load. Finally, a stress test will be done in simulating the extreme conditions of peak usage to validate the platform under such conditions of workload.

Work plan

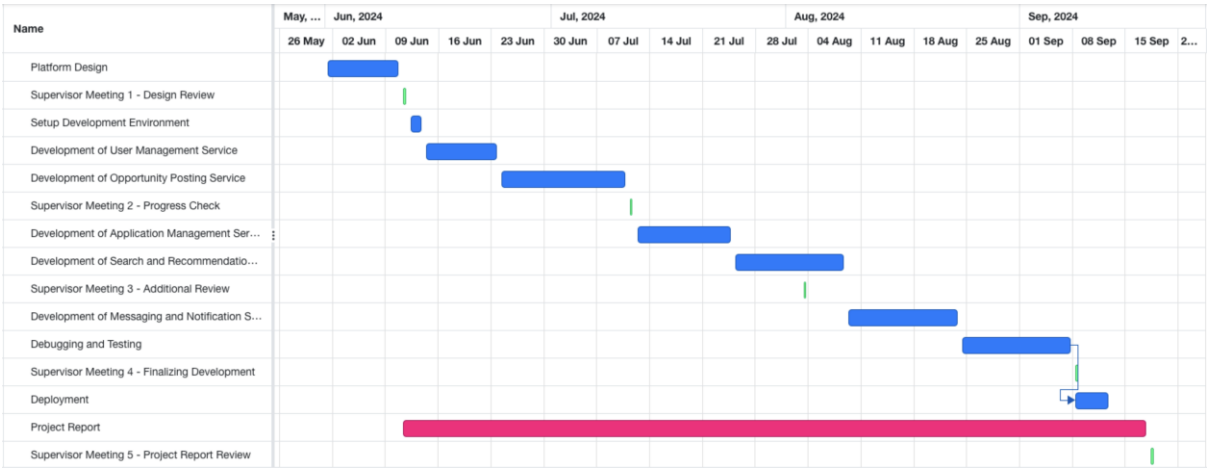


Figure 9 - Gantt Diagram

Risks

Risk	Likelihood (1-3)	Consequences (1-5)	Impact (L x C)	Mitigation Strategies
Data Breach or Privacy Violation	2	4	8	Implement encryption for data at rest and in transit. Compliance checks with GDPR and other relevant regulations.
Service Downtime	2	4	8	Regular backup of all data and system configurations using Spanner capabilities in this field.
Scalability Limitations	1	3	3	Design the system with scalable architecture from the outset. Regular load testing and performance tuning.

Inaccurate Data Handling	2	4	8	Implement data validation at various input points.
Code is lost	2	5	10	Use version control software e.g. GitHub
Resource Constraints	3	3	9	Detailed project planning. There is buffer time allocated in the end of development and every step of it.

Table 10 – Risks Register

References

[1] Bondar, O., 2023. Important LinkedIn Statistics, Data & Trends. *LinkedIn Pulse*, [online] Available at: https://www.linkedin.com/pulse/important-linkedin-statistics-data-trends-oleksii-bondar-pqlie/ [Accessed 14 May 2024].

[2] LinkedIn, 2024. *About Us. LinkedIn Statistics*, [online] Available at: https://news.linkedin.com/about-us [Accessed 14 May 2024].

[3] Indeed, 2024. *About Indeed. Indeed Statistics*, [online] Available at: https://uk.indeed.com/about [Accessed 14 May 2024].

[4] Ofcom, 2022. Adults' Media Use and Attitudes 2022: Interactive Report. Ofcom Report, [online] Available at:

https://www.ofcom.org.uk/__data/assets/pdf_file/0028/241275/Adults-Media-Use-and-Attitudes-report-2022.pdf [Accessed 14 May 2024].

[5] Wuchty, S., Jones, B.F. and Uzzi, B., 2007. *The Increasing Dominance of Teams in Production of Knowledge. Science*, [online] 316(5827), pp.1036-1039. Available at: https://typeset.io/papers/the-increasing-dominance-of-teams-in-production-of-knowledge-h591ohwfzk [Accessed 5 May 2024].

[6] Xiao, X. and Prompanyo, M., 2021. *Influence of Scientific Collaboration Network on Academic Performance.* The International Journal of Business and Management, [online] 16(12), pp.68-. Available at: https://doi.org/10.5539/IJBM.V16N12P68 [Accessed 6 May 2024].

[7] Business of Apps, 2024. LinkedIn Statistics. *Business of Apps*, [online] Available at: https://www.businessofapps.com/data/linkedin-statistics/ [Accessed 15 May 2024].

[8] Boden, R. and Epstein, D., 2006. *Opportunities in the Post-Academic World. Studies in Higher Education*, [online] 31(5), pp.569-585. Available at: https://typeset.io/papers/opportunities-in-the-post-academic-world-3w6m4w6j7l [Accessed 2 May 2024].

[9] International Labour Organization, 2020. *World Employment and Social Outlook: Trends 2020*. ILO Report, [online] Available at: https://typeset.io/papers/world-employment-and-social-outlook-trends-2020-lmuc4uj2sk [Accessed 1 May 2024].

Research Ethics Review Form for MSc Projects

Computer Science Research Ethics Committee (CSREC)

https://www.city.ac.uk/about/governance/committees/cs-research-ethics

Postgraduate students undertaking their final project in the Department of Computer Science must consider the ethics of their project work and ensure that it complies with research ethics guidelines and the law for data protection. In some cases, a project will need approval from an ethics committee before it can proceed. Usually, but not always, this will be because the student is involving other people ("participants") in the project.

To ensure that they give appropriate consideration to ethical issues, all students must complete this form and attach it to their project proposal document. There are two parts:

PART A: Ethics Checklist. All students must complete this part.

The checklist identifies whether the project requires ethical approval and, if so, where to apply for approval.

PART B: Ethics Proportionate Review Form. Students who have answered "no" to all questions in A1, A2 and A3 and "yes" to question 4 in A4 in the ethics checklist must complete part B as well. The project supervisor has delegated authority to provide approval in such cases that are considered to involve MINIMAL risk. The approval may be ***provisional*** – *identifying the planned work with human end user participants as likely* to involve MINIMAL RISK. In such cases you must additionally seek ***full approval*** from the supervisor as the project progresses and details are established. ***Full approval*** must be acquired in writing, before recruiting and engaging with human end users participants for your project.

A.1 If you answer YES to any of the questions in this block, you must apply to an appropriate external ethics committee for approval and log this approval as an External Application through Research Ethics Online - https://researchmanager.city.ac.uk/. This type of research is not covered by City's process, and external approval from an appropriate institution is required.		*Delete as appropriate*
1.1	Does your research require approval from the National Research Ethics Service (NRES)?	**NO**
1.2	Will you recruit participants who are covered by the Mental Capacity Act 2005?	**NO**
1.3	Will you recruit any participants who are covered by the Criminal Justice System, for example, people on remand, prisoners and those on probation?	**NO**
A.2 If you answer YES to any of the questions in this block, then unless you are applying to an external ethics committee, you must apply for approval from the Senate Research Ethics Committee (SREC) through Research Ethics Online - https://researchmanager.city.ac.uk/		*Delete as appropriate*
2.1	Does your research involve participants who are unable to give informed consent?	**NO**
2.2	Is there a risk that your research might lead to disclosures from participants concerning their involvement in illegal activities?	**NO**
2.3	Is there a risk that obscene and or illegal material may need to be accessed for your research study (including online content and other material)?	**NO**
2.4	Does your project involve participants disclosing information about protected characteristics (as identified by the Equality Act 2010)?	**NO**
2.5	Does your research involve you travelling to another country outside of the UK, where the Foreign & Commonwealth Office has issued a travel warning that affects the area in which you will study?	**NO**
2.6	Does your research involve invasive or intrusive procedures?	**NO**
2.7	Does your research involve animals?	**NO**

2.8	Does your research involve the administration of drugs, placebos or other substances to study participants?	**NO**
A.3 If you answer YES to any of the questions in this block, then unless you are applying to an external ethics committee or the Senate Research Ethics Committee (SREC), you must apply for approval from the Computer Science Research Ethics Committee (CSREC) through Research Ethics Online - https://researchmanager.city.ac.uk/. Depending on the level of risk associated with your application, it may be referred to the Senate Research Ethics Committee.		*Delete as appropriate*
3.1	Does your research involve participants who are under the age of 18?	**NO**
3.2	Does your research involve adults who are vulnerable because of their social, psychological or medical circumstances (vulnerable adults)?	**NO**
3.3	Are participants recruited because they are staff or students of City, University of London?	**NO**
3.4	Does your research involve intentional deception of participants?	**NO**
3.5	Does your research involve participants taking part without their informed consent?	**NO**
3.5	Is the risk posed to participants greater than that in normal working life?	**NO**
3.7	Is the risk posed to you, the researcher(s), greater than that in normal working life?	**NO**

A.4 If you answer YES to the following question and your answers to all other questions in sections A1, A2 and A3 are NO, then your project is deemed to be of MINIMAL RISK. **If this is the case, then you can apply for approval through your supervisor under PROPORTIONATE REVIEW. You do so by completing PART B of this form.** **If you have answered NO to all questions on this form, then your project does not require ethical approval. You should submit and retain this form as evidence of this.**		*Delete as appropriate*
4	Does your project involve human participants or their identifiable personal data?	**NO**

Appendix B – Source Code

B.1 Frontend

Components/Navigation.js

```
import React, { useContext } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import './Navigation.css';
import { AuthContext } from '../AuthContext.js';

function Navigation() {
  const { isLoggedIn, logout } = useContext(AuthContext);
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate('/login');
  };

  return (
    <nav>
      <ul>
        {isLoggedIn ? (
          <>
            <li>
              <Link to="/profile">Profile</Link>
            </li>
            <li>
              <Link to="/post-opportunity">Post Opportunity</Link>
            </li>
            <li>
              <Link to="/search">Search Opportunities</Link>
            </li>
            <li>
            <Link to="/conversations">Messages</Link>
            </li>
            <li>
              <button onClick={handleLogout}>Logout</button>
            </li>
          </>
        ) : (
          <>
            <li>
              <Link to="/login">Login</Link>
            </li>
            <li>
              <Link to="/register">Register</Link>
            </li>
          </>
        )}
      </ul>
    </nav>
```

```
  );
}

export default Navigation;
```

components/Navigation.css

```
nav {
  background-color: #333;
  overflow: hidden;
}

nav ul {
  list-style-type: none;
  margin: 0;
  padding: 0;
  display: flex;
  justify-content: space-around;
}

nav li a, nav li button {
  display: block;
  color: white;
  background: none;
  border: none;
  cursor: pointer;
  text-align: center;
  padding: 14px 20px;
  text-decoration: none;
  font-size: 16px;
}

nav li a:hover, nav li button:hover {
  background-color: #111;
}

nav li button {
  font-family: inherit;
  font-size: inherit;
  color: white;
}
```

Components/ProtectedRoute.js

```
import React from 'react';
import { Navigate } from 'react-router-dom';

function ProtectedRoute({ children }) {
  const token = localStorage.getItem('token');
  return token ? children : <Navigate to="/login" />;
```

```
}

export default ProtectedRoute;
```

pages/ApplicationFormPage.js

```
import React, { useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import axios from 'axios';

function ApplicationFormPage() {
  const { opportunityID } = useParams();
  const [motivationLetter, setMotivationLetter] = useState('');
  const [cvFile, setCvFile] = useState(null);
  const [error, setError] = useState('');
  const [successMessage, setSuccessMessage] = useState('');
  const navigate = useNavigate();

  const handleFileChange = (e) => {
    setCvFile(e.target.files[0]);
  };

  const handleApply = async (e) => {
    e.preventDefault();

    const token = localStorage.getItem('token');
    if (!token) {
      setError('You need to log in to apply');
      return navigate('/login');
    }

    //object for handling the files
    const formData = new FormData();
    formData.append('motivationLetter', motivationLetter);
    if (cvFile) {
      formData.append('cv', cvFile);
    }

    try {
      const response = await axios.post(
        `${process.env.REACT_APP_BACKEND_OPPORTUNITY_URL}/api/opportunities/${opportunityID}/apply`,
        formData,
        {
          headers: {
            Authorization: `Bearer ${token}`,
            'Content-Type': 'multipart/form-data',
          },
        }
      );

      setSuccessMessage('Application submitted successfully!');
```

```
      setError('');
      setMotivationLetter('');
      setCvFile(null);
    } catch (error) {
      setError('Failed to submit application. Please try again.');
    }
  };

  return (
    <div>
      <h1>Apply for Opportunity</h1>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      {successMessage && <p style={{ color: 'green' }}>{successMessage}</p>}
      <form onSubmit={handleApply}>
        <div>
          <label>Motivation Letter</label>
          <textarea
            value={motivationLetter}
            onChange={(e) => setMotivationLetter(e.target.value)}
            required
          ></textarea>
        </div>
        <div>
          <label>Upload CV </label>
          <input type="file" accept=".pdf,.doc,.docx" onChange={handleFileChange} />
        </div>
        <button type="submit">Submit Application</button>
      </form>
    </div>
  );
}

export default ApplicationFormPage;
```

pages/ApplicationsByOppPage.js

```
import React, { useEffect, useState } from 'react';
import { useParams } from 'react-router-dom';
import axios from 'axios';

function ApplicationsByOppPage() {
  const { id } = useParams();
  const [applications, setApplications] = useState([]);
  const [error, setError] = useState('');
  const creatorID = localStorage.getItem('userID');

  useEffect(() => {
    const fetchApplications = async () => {
      const token = localStorage.getItem('token');

      if (!token) {
```

```
        setError('Authorization required');
        return;
      }

      try {
        const response = await
axios.get(`${process.env.REACT_APP_BACKEND_OPPORTUNITY_URL}/api/opportunities/${id}/applications`, {
          headers: {
            Authorization: `Bearer ${token}`,
          },
        });
        setApplications(response.data);
      } catch (error) {
        setError('Failed to fetch applications');
      }
    };

    fetchApplications();
  }, [id]);

  const handleCreateConversation = async (applicantID, opportunityID, creatorID) => {
    const token = localStorage.getItem('token');

    try {
      await axios.post(`${process.env.REACT_APP_BACKEND_MESSAGING_URL}/api/conversations`, {
        id,
        applicantID,
        creatorID,
        opportunityID
      }, {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      });
      alert('Conversation created successfully');
    } catch (error) {
      alert('Failed to create conversation');
    }
  };

  return (
    <div>
      <h1>Applications for Opportunity</h1>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      {applications.length > 0 ? (
        <ul>
          {applications.map((application) => (
            <li key={application.ApplicationID}>
              <p><strong>User ID:</strong> {application.UserID}</p>
              <p><strong>Application Date:</strong> {new Date(application.ApplicationDate).toLocaleString()}</p>
              <p><strong>Motivation Letter:</strong> {application.MotivationLetter}</p>
```

```
            {application.CVUrl && (
              <p>
                <strong>CV:</strong> <a href={application.CVUrl} target="_blank" rel="noopener
noreferrer">Download CV</a>
              </p>
            )}
            <button onClick={() => handleCreateConversation(application.UserID, application.OpportunityID,
creatorID)}>Message</button>
          </li>
        ))}
      </ul>
    ) : (
      !error && <p>No applications found</p>
    )}
  </div>
 );
}

export default ApplicationsByOppPage;
```

pages/ApplicationsByUserPage.js

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

function ApplicationsByUserPage() {
 const [applications, setApplications] = useState([]);
 const [error, setError] = useState('');

 useEffect(() => {
  const fetchMyApplications = async () => {
   const token = localStorage.getItem('token');

   if (!token) {
    setError('Authorization required');
    return;
   }

   // Getting the user applications
   try {
    const response = await axios.get(`${process.env.REACT_APP_BACKEND_OPPORTUNITY_URL}/api/my-
applications`, {
     headers: {
      Authorization: `Bearer ${token}`,
     },
    });

    // Getting opportunity details for each application
    const applicationsWithDetails = await Promise.all(
     response.data.map(async (application) => {
```

```
          const opportunityResponse = await
axios.get(`${process.env.REACT_APP_BACKEND_OPPORTUNITY_URL}/api/opportunities/${application.Opport
unityID}`, {
            headers: {
              Authorization: `Bearer ${token}`,
            },
          });

          return {
            ...application,
            OpportunityTitle: opportunityResponse.data.Title,
          };
        })
      );

      setApplications(applicationsWithDetails);
    } catch (error) {
      setError('Failed to fetch your applications');
    }
  };

  fetchMyApplications();
 }, []);

 return (
  <div>
    <h1>My Applications</h1>
    {error && <p style={{ color: 'red' }}>{error}</p>}
    {applications.length > 0 ? (
      <ul>
        {applications.map((application) => (
          <li key={application.ApplicationID}>
            <p><strong>Opportunity:</strong> {application.OpportunityTitle}</p>
            <p><strong>Application Date:</strong> {new Date(application.ApplicationDate).toLocaleString()}</p>
            <p><strong>Motivation Letter:</strong> {application.MotivationLetter}</p>
            {application.CVUrl && (
              <p>
                <strong>CV:</strong> <a href={application.CVUrl} target="_blank" rel="noopener
noreferrer">Download CV</a>
              </p>
            )}
          </li>
        ))}
      </ul>
    ) : (
      !error && <p>You have not submitted any applications yet</p>
    )}
  </div>
 );
}
```

```
export default ApplicationsByUserPage;
```

pages/ChatPage.js

```
import React, { useEffect, useState } from 'react';
import { useParams } from 'react-router-dom';
import axios from 'axios';
import {jwtDecode} from 'jwt-decode';

const ChatPage = () => {
  const { conversationID } = useParams();
  const [messages, setMessages] = useState([]);
  const [newMessage, setNewMessage] = useState('');
  const [error, setError] = useState('');

  useEffect(() => {
    const fetchMessages = async () => {
      const token = localStorage.getItem('token');
      if (!token) {
        setError('Authorization required');
        return;
      }

      try {
        const response = await
axios.get(`${process.env.REACT_APP_BACKEND_MESSAGING_URL}/api/conversations/${conversationID}/me
ssages`, {
          headers: {
            Authorization: `Bearer ${token}`,
          },
        });
        setMessages(Array.isArray(response.data) ? response.data : []);
      } catch (error) {
        setError('Failed to fetch messages');
      }
    };

    fetchMessages();
  }, [conversationID]);

  const handleSendMessage = async () => {
    const token = localStorage.getItem('token');
    if (!newMessage.trim()) return;

    try {
      await
axios.post(`${process.env.REACT_APP_BACKEND_MESSAGING_URL}/api/conversations/${conversationID}/m
essages`, {
        senderID: jwtDecode(token).userId,
        messageContent: newMessage,
      }, {
        headers: {
```

```
        Authorization: `Bearer ${token}`,
      },
    });
    setNewMessage('');
    window.location.reload();
  } catch (error) {
    setError('Failed to send message');
  }
};




 return (
  <div>
    <h1>Chat</h1>
    {error && <p style={{ color: 'red' }}>{error}</p>}
      <ul>
      {messages.length > 0 ? (
      messages.map((message, index) => (
          <li key={index}>
              <strong>{message.SenderName || 'Unknown'}:</strong> {message.MessageContent}
          </li>
          ))
      ) : (
          <li>No messages yet.</li>
          )}
      </ul>

    <input
      type="text"
      value={newMessage}
      onChange={(e) => setNewMessage(e.target.value)}
    />
    <button onClick={handleSendMessage}>Send</button>
  </div>
 );
};

export default ChatPage;
```

pages/ConversationsPage.js

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';
import { Link, useNavigate } from 'react-router-dom';

const ConversationsPage = () => {
  const [conversations, setConversations] = useState([]);
  const [error, setError] = useState('');
  const navigate = useNavigate(); // To navigate to a conversation
```

```
  const fetchConversations = async () => {
    const token = localStorage.getItem('token');
    if (!token) {
      setError('Authorization required');
      return;
    }

    try {
      const response = await
axios.get(`${process.env.REACT_APP_BACKEND_MESSAGING_URL}/api/conversations`, {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      });
      setConversations(response.data);
    } catch (error) {
      setError('Failed to fetch conversations');
    }
  };

  // Fetch conversations on component mount
  useEffect(() => {
    fetchConversations();
  }, []);

  // Function to handle creating a new conversation
  const handleCreateConversation = async (applicantID, opportunityID) => {
    const token = localStorage.getItem('token');

    try {
      const response = await
axios.post(`${process.env.REACT_APP_BACKEND_MESSAGING_URL}/api/conversations`, {
        opportunityID,
        applicantID,
      }, {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      });

      const { conversationID } = response.data;
      alert('Conversation created successfully!');
      navigate(`/conversations/${conversationID}`);
      fetchConversations();
    } catch (error) {
      alert('Failed to create conversation');
    }
  };

  return (
    <div>
```

```
      <h1>Your Conversations</h1>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      {conversations.length > 0 ? (
        <ul>
          {conversations.map((conv) => (
            <li key={conv.ConversationID}>
              <p>Opportunity ID: {conv.OpportunityID}</p>
              <p>Last Message: {conv.LastMessageTimestamp}</p>
              <Link to={`/conversations/${conv.ConversationID}`}>Open Chat</Link>
            </li>
          ))}
        </ul>
      ) : (
        !error && <p>No conversations yet</p>
      )}
    </div>
  );
};

export default ConversationsPage;
```

pages/EditOpportunityPage.js

```
import React, { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import axios from 'axios';

function EditOpportunityPage() {
  const { id } = useParams();
  const navigate = useNavigate();
  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  const [location, setLocation] = useState('');
  const [deadline, setDeadline] = useState('');
  const [educationLevel, setEducationLevel] = useState('');
  const [subjectFilters, setSubjectFilters] = useState('');
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchOpportunity = async () => {
      const token = localStorage.getItem('token');
      if (!token) return navigate('/login');

      try {
        const response = await
axios.get(`${process.env.REACT_APP_BACKEND_OPPORTUNITY_URL}/api/opportunities/${id}`, {
          headers: { Authorization: `Bearer ${token}` },
        });
        const opp = response.data;
        setTitle(opp.Title);
        setDescription(opp.Description);
```

```
      setLocation(opp.Location);
      setDeadline(opp.Deadline);
      setEducationLevel(opp.EducationLevel);
      setSubjectFilters(opp.SubjectFilters.join(', '));
    } catch (err) {
      setError('Failed to fetch opportunity');
    }
  };

  fetchOpportunity();
}, [id, navigate]);

const handleSubmit = async (e) => {
  e.preventDefault();
  const token = localStorage.getItem('token');
  try {
    await axios.put(`${process.env.REACT_APP_BACKEND_OPPORTUNITY_URL}/api/opportunities/${id}`, {
      title,
      description,
      location,
      deadline,
      educationLevel,
      subjectFilters: subjectFilters.split(',').map(f => f.trim()),
    }, {
      headers: { Authorization: `Bearer ${token}` },
    });
    navigate('/opportunities');
  } catch (err) {
    setError('Failed to update opportunity');
  }
};

return (
  <div>
    <h1>Edit Opportunity</h1>
    {error && <p style={{ color: 'red' }}>{error}</p>}
    <form onSubmit={handleSubmit}>
      <div>
        <label>Title</label>
        <input type="text" value={title} onChange={e => setTitle(e.target.value)} required />
      </div>
      <div>
        <label>Description</label>
        <textarea value={description} onChange={e => setDescription(e.target.value)} required />
      </div>
      <div>
        <label>Location</label>
        <input type="text" value={location} onChange={e => setLocation(e.target.value)} />
      </div>
      <div>
        <label>Deadline</label>
```

```
        <input type="date" value={deadline} onChange={e => setDeadline(e.target.value)} required />
      </div>
      <div>
        <label>Education Level</label>
        <input type="text" value={educationLevel} onChange={e => setEducationLevel(e.target.value)} />
      </div>
      <div>
        <label>Subject Filters (comma-separated)</label>
        <input type="text" value={subjectFilters} onChange={e => setSubjectFilters(e.target.value)} />
      </div>
      <button type="submit">Update Opportunity</button>
    </form>
  </div>
 );
}

export default EditOpportunityPage;
```

pages/HomePage.js

```
import React from 'react';

function HomePage() {
 return (
  <div>
   <h1>Home</h1>
   <p>Welcome to the application</p>
  </div>
 );
}

export default HomePage;
```

pages/LoginPage.js

```
import React, { useState, useContext } from 'react';
import { GoogleOAuthProvider, GoogleLogin } from '@react-oauth/google';
import { useNavigate } from 'react-router-dom';
import axios from 'axios';
import { AuthContext } from '../AuthContext';


const clientId = process.env.REACT_APP_GOOGLE_CLIENT_ID;

function LoginPage() {
 const [email, setEmail] = useState('');
 const [password, setPassword] = useState('');
 const [error, setError] = useState(null);
 const navigate = useNavigate();
 const { login } = useContext(AuthContext);

 const handleLogin = async (e) => {
```

```
    e.preventDefault();
    try {
      const response = await axios.post(`${process.env.REACT_APP_BACKEND_USER_URL}/api/auth/login`, {
        email,
        password,
      });
      login(response.data.token);
      navigate('/profile');
    } catch (error) {
      setError(error.response ? error.response.data.message : 'Login failed');
    }
  };

  const handleGoogleLogin = async (response) => {
    try {
      const response = await axios.post(`${process.env.REACT_APP_BACKEND_USER_URL}/api/auth/google`, {
        tokenId: response.credential,
      });
      login(response.data.token);
      navigate('/profile');
    } catch (error) {
      setError('Google login failed');
    }
  };

  return (
    <GoogleOAuthProvider clientId={clientId}>
      <div>
        <h1>Login</h1>
        <form onSubmit={handleLogin}>
          <div>
            <label>Email</label>
            <input
              type="email"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              required
            />
          </div>
          <div>
            <label>Password</label>
            <input
              type="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              required
            />
          </div>
          {error && <p style={{ color: 'red' }}>{error}</p>}
          <button type="submit">Login</button>
        </form>
```

```
        <div>
          <h2>Or</h2>
          <GoogleLogin
            onSuccess={handleGoogleLogin}
            onError={() => {
              setError('Google login failed');
            }}
          />
        </div>
      </div>
    </GoogleOAuthProvider>
  );
}

export default LoginPage;
```

pages/OpportunityDetailPage.js

```
import React, { useEffect, useState } from 'react';
import { useParams, Link } from 'react-router-dom';
import axios from 'axios';

function OpportunityDetailPage() {
  const { opportunityID } = useParams();
  const [opportunity, setOpportunity] = useState(null);
  const [error, setError] = useState('');
  const [isCreator, setIsCreator] = useState(false);

  useEffect(() => {
    const fetchOpportunityDetails = async () => {
      const token = localStorage.getItem('token');

      if (!token) {
        setError('Authorization required');
        return;
      }

      // Getting opportunity details
      try {
        const response = await
axios.get(`${process.env.REACT_APP_BACKEND_OPPORTUNITY_URL}/api/opportunities/${opportunityID}`, {
          headers: {
            Authorization: `Bearer ${token}`,
          },
        });

        setOpportunity(response.data);

        // Getting the current usr
        const userProfile = await axios.get(`${process.env.REACT_APP_BACKEND_USER_URL}/api/auth/profile`,
{
```

```
        headers: {
          Authorization: `Bearer ${token}`,
        },
      });

      // Checking if the user iscreator of the opportunity
      setIsCreator(userProfile.data.id === response.data.PostedBy);
    } catch (error) {
      console.error('Error fetching opportunity details or user profile:', error);
      setError('Failed to fetch opportunity details');
    }
  };

  fetchOpportunityDetails();
}, [opportunityID]);

return (
  <div>
    <h1>Opportunity Details</h1>
    {error && <p style={{ color: 'red' }}>{error}</p>}
    {opportunity ? (
      <div>
        <h2>{opportunity.Title}</h2>
        <p>{opportunity.Description}</p>
        <p><strong>Location:</strong> {opportunity.Location}</p>
        <p><strong>Deadline:</strong> {opportunity.Deadline}</p>
        <p><strong>Education Level:</strong> {opportunity.EducationLevel}</p>
        <p><strong>Subjects:</strong> {opportunity.SubjectFilters.join(', ')}</p>

        {!isCreator && (
          <Link to={`/opportunities/${opportunity.OpportunityID}/apply`}>
            <button>Apply for this Opportunity</button>
          </Link>
        )}
        {isCreator && (
          <div style={{ marginTop: '20px' }}>
            <Link to={`/opportunities/${opportunity.OpportunityID}/applications`}>
              <button>View Applications for this Opportunity</button>
            </Link>
          </div>
        )}
      </div>
    ) : (
      !error && <p>Loading opportunity details...</p>
    )}
    {error === 'Failed to fetch opportunity details' && (
      <div>
        <h2>404: Opportunity Not Found</h2>
        <p>The opportunity you're looking for does not exist or you do not have access to it.</p>
      </div>
    )}
```

```
    </div>
  );
}

export default OpportunityDetailPage;
```

pages/OpportunityListPage.js

```
import React, { useEffect, useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import axios from 'axios';

function OpportunityListPage() {
  const [opportunities, setOpportunities] = useState([]);
  const [error, setError] = useState(null);
  const navigate = useNavigate();

  useEffect(() => {
    const fetchOpportunities = async () => {
      const token = localStorage.getItem('token');
      if (!token) return navigate('/login');

      try {
        const response = await
axios.get(`${process.env.REACT_APP_BACKEND_OPPORTUNITY_URL}/api/opportunities`, {
          headers: { Authorization: `Bearer ${token}` },
        });
        setOpportunities(response.data);
      } catch (err) {
        setError('Failed to fetch opportunities');
      }
    };

    fetchOpportunities();
  }, [navigate]);

  const handleDelete = async (id) => {
    const token = localStorage.getItem('token');
    try {
      await axios.delete(`${process.env.REACT_APP_BACKEND_OPPORTUNITY_URL}/api/opportunities/${id}`, {
        headers: { Authorization: `Bearer ${token}` },
      });
      setOpportunities(opportunities.filter(opp => opp.OpportunityID !== id));
    } catch (err) {
      setError('Failed to delete opportunity');
    }
  };

  return (
    <div>
      <h1>Your Opportunities</h1>
```

```
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <ul>
        {opportunities.map(opp => (
          <li key={opp.OpportunityID}>
            <h3>{opp.Title}</h3>
            <p>{opp.Description}</p>
            <Link to={`/opportunities/${opp.OpportunityID}`}>View</Link> |
            <Link to={`/opportunities/edit/${opp.OpportunityID}`}>Edit</Link> |
            <button onClick={() => handleDelete(opp.OpportunityID)}>Delete</button>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default OpportunityListPage;
```

Pages/PostOpportunityPage.js

```
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import axios from 'axios';

function PostOpportunityPage() {
  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  const [location, setLocation] = useState('');
  const [deadline, setDeadline] = useState('');
  const [educationLevel, setEducationLevel] = useState('');
  const [subjectFilters, setSubjectFilters] = useState('');
  const [error, setError] = useState(null);
  const [successMessage, setSuccessMessage] = useState('');
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();

    const token = localStorage.getItem('token');
    if (!token) {
      setError('You need to log in to post an opportunity');
      return navigate('/login');
    }

    const subjectArray = subjectFilters.split(',').map(subject => subject.trim());
```

```
    try {
      const response = await
axios.post(`${process.env.REACT_APP_BACKEND_OPPORTUNITY_URL}/api/opportunities`, {
        title,
        description,
        location,
        deadline,
        educationLevel,
        subjectFilters: subjectArray,
      }, {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      });

      setSuccessMessage('Opportunity posted successfully!');
      setError(null);
      setTitle('');
      setDescription('');
      setLocation('');
      setDeadline('');
      setEducationLevel('');
      setSubjectFilters('');
    } catch (error) {
      setError('Failed to post opportunity. Please try again.');
    }
  };

  return (
    <div>
      <h1>Post Opportunity</h1>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      {successMessage && <p style={{ color: 'green' }}>{successMessage}</p>}
      <form onSubmit={handleSubmit}>
        <div>
          <label>Title</label>
          <input
            type="text"
            value={title}
            onChange={(e) => setTitle(e.target.value)}
            required
          />
        </div>
        <div>
          <label>Description</label>
          <textarea
            value={description}
            onChange={(e) => setDescription(e.target.value)}
            required
          ></textarea>
        </div>
```

```
        <div>
          <label>Location</label>
          <input
            type="text"
            value={location}
            onChange={(e) => setLocation(e.target.value)}
            required
          />
        </div>
        <div>
          <label>Deadline</label>
          <input
            type="date"
            value={deadline}
            onChange={(e) => setDeadline(e.target.value)}
            required
          />
        </div>
        <div>
          <label>Education Level</label>
          <input
            type="text"
            value={educationLevel}
            onChange={(e) => setEducationLevel(e.target.value)}
            required
          />
        </div>
        <div>
          <label>Subject Filters (comma separated)</label>
          <input
            type="text"
            value={subjectFilters}
            onChange={(e) => setSubjectFilters(e.target.value)}
            placeholder="e.g., Math, Science, Engineering"
            required
          />
        </div>
        <button type="submit">Post Opportunity</button>
      </form>
    </div>
  );
}

export default PostOpportunityPage;
```

pages/ProfilePage.js

```
import React, { useEffect, useState } from 'react';
import { getProfile } from '../services/authService';
import { useNavigate, Link } from 'react-router-dom';
```

```
function ProfilePage() {
  const [profile, setProfile] = useState(null);
  const [error, setError] = useState(null);
  const navigate = useNavigate();

  useEffect(() => {
    const fetchProfile = async () => {
      try {
        const token = localStorage.getItem('token');

        if (!token) {
          navigate('/login');
          return;
        }

        const data = await getProfile(token);
        setProfile(data);
      } catch (error) {
        if (error.response && error.response.status === 401) {
          localStorage.removeItem('token');
          navigate('/login');
        } else {
          setError(error.response ? error.response.data.message : 'Failed to fetch profile');
        }
      }
    };

    fetchProfile();
  }, [navigate]);

  return (
    <div>
      <h1>User Profile</h1>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      {profile ? (
        <div>
          <p><strong>Name:</strong> {profile.name}</p>
          <p><strong>Email:</strong> {profile.email}</p>
          <Link to="/opportunities">
            <button>View Your Opportunities</button>
          </Link>
          <br/>
          <br/>
          <Link to="/my-applications">
            <button>View Opportunities You Applied For</button>
          </Link>
        </div>
      ) : (
        !error && <p>Loading profile...</p>
      )}
    </div>
```

```
  );
}

export default ProfilePage;
```

pages/RegisterPage.js

```
import React, { useState } from 'react';
import { register } from '../services/authService';
import { useNavigate } from 'react-router-dom';

function RegisterPage() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState(null);
  const navigate = useNavigate();

  const handleRegister = async (e) => {
    e.preventDefault();
    try {
      const data = await register(name, email, password);
      localStorage.setItem('token', data.token);
      navigate('/profile');
    } catch (error) {
      setError(error.response.data.message);
    }
  };

  return (
    <div>
      <h1>Register</h1>
      <form onSubmit={handleRegister}>
        <div>
          <label>Name</label>
          <input
            type="text"
            value={name}
            onChange={(e) => setName(e.target.value)}
            required
          />
        </div>
        <div>
          <label>Email</label>
          <input
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
          />
        </div>
```

```
        <div>
          <label>Password</label>
          <input
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
          />
        </div>
        {error && <p>{error}</p>}
        <button type="submit">Register</button>
      </form>
    </div>
  );
}

export default RegisterPage;
```

Pages/SearchPage.js

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import axios from 'axios';

function SearchPage() {
  const [title, setTitle] = useState('');
  const [location, setLocation] = useState('');
  const [educationLevel, setEducationLevel] = useState('');
  const [subjectFilters, setSubjectFilters] = useState([]);
  const [results, setResults] = useState([]);
  const [error, setError] = useState('');

  useEffect(() => {
    const fetchAllOpportunities = async () => {
      try {
        const response = await
axios.get(`${process.env.REACT_APP_BACKEND_SEARCH_URL}/api/opportunities`);
        setResults(response.data);
        setError('');
      } catch (error) {
        setResults([]);
        setError('Error loading opportunities');
      }
    };

    fetchAllOpportunities();
  }, []);
```

```
  const handleSearch = async (e) => {
    e.preventDefault();
    try {
      const response = await
axios.get(`${process.env.REACT_APP_BACKEND_SEARCH_URL}/api/opportunities/search`, {
        params: {
          title,
          location,
          educationLevel,
          subjectFilters: subjectFilters.join(',')
        },
      });
      setResults(response.data);
      setError('');
    } catch (error) {
      setResults([]);
      setError('No matching opportunities found');
    }
  };

  const handleSubjectFilterChange = (e) => {
    const { options } = e.target;
    const selectedFilters = Array.from(options)
      .filter(option => option.selected)
      .map(option => option.value);
    setSubjectFilters(selectedFilters);
  };

  const handleClearFilters = () => {
    setTitle('');
    setLocation('');
    setEducationLevel('');
    setSubjectFilters([]);
    window.location.reload();
  };

  return (
    <div className="search-page">
      <div className="filters">
        <h2>Filters</h2>
        <form onSubmit={handleSearch}>
          <div>
            <label>Title</label>
            <input type="text" value={title} onChange={(e) => setTitle(e.target.value)} />
          </div>
          <div>
            <label>Location</label>
            <input type="text" value={location} onChange={(e) => setLocation(e.target.value)} />
          </div>
          <div>
```

```
          <label>Education Level</label>
          <input type="text" value={educationLevel} onChange={(e) => setEducationLevel(e.target.value)} />
        </div>
        <div>
          <label>Subject Filters (hold CTRL/Command for multiple)</label>
          <select multiple onChange={handleSubjectFilterChange} value={subjectFilters}>
            <option value="Math">Math</option>
            <option value="Computer Science">Computer Science</option>
            <option value="Software Engineering">Software Engineering</option>
            <option value="Data Science">Data Science</option>
            <option value="Artificial Intelligence">Artificial Intelligence</option>
          </select>
        </div>
        <button type="submit">Search</button>
        <button type="button" onClick={handleClearFilters} style={{ marginLeft: '10px' }}>
          Clear Filters
        </button>
      </form>
    </div>

    <div className="results">
      <h2>Available Opportunities</h2>
      {error && <p style={{ color: 'red' }}>{error}</p>}

      {results.length > 0 ? (
        <ul>
          {results.map((opportunity) => (
            <li key={opportunity.OpportunityID}>
              <h3>
                <Link to={`/opportunities/${opportunity.OpportunityID}`}>
                  {opportunity.Title}
                </Link>
              </h3>
              <p>{opportunity.Description}</p>
              <p><strong>Deadline:</strong> {opportunity.Deadline}</p>
            </li>
          ))}
        </ul>
      ) : (
        !error && <p>No opportunities found</p>
      )}
    </div>
  </div>
 );
}

export default SearchPage;
```

App.js

```
import React from 'react';
```

```
import './App.css';
import AppRouter from './Router';
import { AuthProvider } from './AuthContext';

function App() {
  return (
    <div className="App">
      <AuthProvider>
        <AppRouter />
      </AuthProvider>
    </div>
  );
}

export default App;
```

App.css

```
.App {
  text-align: center;
}

.App-logo {
  height: 40vmin;
  pointer-events: none;
}

@media (prefers-reduced-motion: no-preference) {
  .App-logo {
    animation: App-logo-spin infinite 20s linear;
  }
}

.App-header {
  background-color: #282c34;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  font-size: calc(10px + 2vmin);
  color: white;
}

.App-link {
  color: #61dafb;
}

@keyframes App-logo-spin {
  from {
    transform: rotate(0deg);
```

```
  }
  to {
    transform: rotate(360deg);
  }
}
.search-bar {
  margin: 20px 0;
  display: flex;
  gap: 10px;
}

.search-results ul,
.recommendations ul {
  list-style-type: none;
  padding: 0;
}

.search-results li,
.recommendations li {
  margin-bottom: 15px;
}

.search-results h2,
.recommendations h2 {
  margin-bottom: 20px;
}
/* App.css */
body {
  font-family: 'Roboto', sans-serif;
  background-color: #f9f9f9;
  margin: 0;
  padding: 0;
}

h1, h2 {
  font-weight: 600;
  color: #333;
}

h1 {
  margin-bottom: 20px;
}

h2 {
  margin-top: 15px;
  margin-bottom: 10px;
  color: #555;
}

p {
  color: #666;
```

```
  line-height: 1.6;
}

.container {
  width: 85%;
  max-width: 1000px;
  margin: 0 auto;
  padding: 20px;
  background-color: white;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

button {
  background-color: #3498db;
  color: white;
  padding: 10px 15px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  transition: background-color 0.3s ease;
}

button:hover {
  background-color: #2980b9;
}

a {
  color: #3498db;
  text-decoration: none;
  font-weight: bold;
}

a:hover {
  text-decoration: underline;
}

.error-message {
  color: red;
  font-weight: bold;
}

ul {
  list-style-type: none;
  padding: 0;
}

li {
  margin-bottom: 15px;
}
```

```
form {
  margin-top: 20px;
}

.form-group {
  margin-bottom: 15px;
}

label {
  font-weight: bold;
}

input[type="text"], input[type="date"], textarea {
  width: 100%;
  padding: 10px;
  margin-top: 5px;
  border: 1px solid #ccc;
  border-radius: 5px;
  box-sizing: border-box;
}

textarea {
  resize: vertical;
}

footer {
  margin-top: 40px;
  text-align: center;
  color: #aaa;
  font-size: 14px;
}
```

Services/authService.js

```
import axios from 'axios';

const API_URL = process.env.REACT_APP_BACKEND_USER_URL || 'http://localhost:5001';

export async function login(email, password) {
  const response = await axios.post(`${API_URL}/api/auth/login`, { email, password });
  return response.data;
}

export async function register(name, email, password) {
  const response = await axios.post(`${API_URL}/api/auth/register`, { name, email, password });
  return response.data;
}

export async function googleLogin(tokenId) {
  const response = await axios.post(`${API_URL}/api/auth/google`, { tokenId });
  return response.data;
```

```
}

export async function getProfile(token) {
  const response = await axios.get(`${API_URL}/api/auth/profile`, {
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });
  return response.data;
}
```

AuthContext.js

```
import React, { createContext, useEffect, useState } from 'react';

export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  useEffect(() => {
    const token = localStorage.getItem('token');
    setIsLoggedIn(!!token);
  }, []);

  const login = (token) => {
    localStorage.setItem('token', token);
    setIsLoggedIn(true);
  };

  const logout = () => {
    localStorage.removeItem('token');
    setIsLoggedIn(false);
  };

  return (
    <AuthContext.Provider value={{ isLoggedIn, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};
```

Router.js

```
import React from 'react';
import { BrowserRouter as Router, Route, Routes } from 'react-router-dom';
import HomePage from './pages/HomePage';
import LoginPage from './pages/LoginPage';
```

```
import RegisterPage from './pages/RegisterPage';
import ProfilePage from './pages/ProfilePage';
import Navigation from './components/Navigation';
import PostOpportunityPage from './pages/PostOpportunityPage';
import OpportunityListPage from './pages/OpportunityListPage';
import EditOpportunityPage from './pages/EditOpportunityPage';
import SearchPage from './pages/SearchPage';
import OpportunityDetailPage from './pages/OpportunityDetailPage';
import ApplicationFormPage from './pages/ApplicationFormPage';
import ApplicationsByOppPage from './pages/ApplicationsByOppPage';
import ApplicationsByUserPage from './pages/ApplicationsByUserPage';
import ConversationsPage from './pages/ConversationsPage';
import ChatPage from './pages/ChatPage';
import ProtectedRoute from './components/ProtectedRoute';

function AppRouter() {
  return (
    <Router>
      <Navigation />
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/login" element={<LoginPage />} />
        <Route path="/register" element={<RegisterPage />} />
        <Route
          path="/post-opportunity"
          element={
            <ProtectedRoute>
              <PostOpportunityPage />
            </ProtectedRoute>
          }
        />
        <Route
          path="/opportunities"
          element={
            <ProtectedRoute>
              <OpportunityListPage />
            </ProtectedRoute>
          }
        />
        <Route
          path="/opportunities/edit/:id"
          element={
            <ProtectedRoute>
              <EditOpportunityPage />
            </ProtectedRoute>
          }
        />
        <Route path="/profile" element={<ProtectedRoute><ProfilePage /></ProtectedRoute>} />
        <Route path="/search" element={<SearchPage />} />
        <Route path="/opportunities/:opportunityID" element={<OpportunityDetailPage />} />
        <Route path="/opportunities/:opportunityID/apply" element={<ApplicationFormPage />} />
```

```
        <Route path="/opportunities/:id/applications" element={<ApplicationsByOppPage />} />
        <Route path="/my-applications" element={<ApplicationsByUserPage />} />
        <Route path="/conversations" element={<ConversationsPage />} />
        <Route path="/conversations/:conversationID" element={<ChatPage />} />
      </Routes>
    </Router>
  );
}

export default AppRouter;
```

B.2 Backend

Messaging-service/index.js

```
const express = require('express');
const cors = require('cors');
const bodyParser = require('body-parser');
const { v4: uuidv4 } = require('uuid');
const axios = require('axios');
const nodemailer = require('nodemailer');
const jwt = require('jsonwebtoken');
const { Spanner } = require('@google-cloud/spanner');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(bodyParser.json());

const spanner = new Spanner({
  projectId: process.env.SPANNER_PROJECT_ID,
});

const instance = spanner.instance(process.env.SPANNER_INSTANCE_ID);
const database = instance.database(process.env.SPANNER_DATABASE_ID);
const usersDatabase = instance.database(process.env.SPANNER_USERS_DATABASE_ID);

// Middleware for JWT authentication
async function authenticateToken(req, res, next) {
  const token = req.headers['authorization'];

  if (!token) {
    return res.status(401).json({ message: 'Authorization token is required' });
  }

  try {
    const bearerToken = token.split(' ')[1];
    const decoded = jwt.verify(bearerToken, process.env.JWT_SECRET);
    req.user = decoded;

    next();
  } catch (error) {
    console.error('Token verification failed:', error);
```

```
    return res.status(403).json({ message: 'Invalid or expired token' });
  }
}

//function to send emails to recipient of the message
async function sendEmailNotification(recipientEmail, subject, messageText) {
  try {
    let transporter = nodemailer.createTransport({
      host: process.env.SMTP_HOST,
      port: process.env.SMTP_PORT,
      secure: true,
      auth: {
        user: process.env.SMTP_USER,
        pass: process.env.SMTP_PASS,
      },
    });
    let info = await transporter.sendMail({
      from: `"Messaging App" <${process.env.SMTP_USER}>`,
      to: recipientEmail,
      subject: subject,
      text: messageText,
    });

    console.log('Email sent: %s', info.messageId);
  } catch (error) {
    console.error('Error sending email:', error);
  }
}

// POST: Create a conversation
app.post('/api/conversations', authenticateToken, async (req, res) => {
  try {
    const conversationID = uuidv4();
    const { opportunityID, applicantID } = req.body;
    const creatorID = req.user.userId;
    const lastMessageTimestamp = new Date().toISOString();
    if (!opportunityID || !creatorID || !applicantID) {
      return res.status(400).json({ message: 'Required fields missing' });
    }
    console.log('Creating conversation with data:', {
      conversationID,
      opportunityID,
      creatorID,
      applicantID,
      lastMessageTimestamp,
    });

    await database.runTransactionAsync(async (transaction) => {
      const query = {
        sql: `INSERT INTO Conversations (ConversationID, OpportunityID, CreatorID, ApplicantID,
LastMessageTimestamp)
```

```
            VALUES (@conversationID, @opportunityID, @creatorID, @applicantID, @lastMessageTimestamp)`,
        params: {
          conversationID,
          opportunityID,
          creatorID,
          applicantID,
          lastMessageTimestamp,
        },
      };

      await transaction.runUpdate(query);
      await transaction.commit();
      res.status(200).json({ message: 'Conversation created successfully!', conversationID });
    });
  } catch (err) {
    console.error('Error creating conversation:', err);
    res.status(500).json({ message: 'Internal Server Error' });
  }
});


 // GET: Getting all conversations for the logged-in user
app.get('/api/conversations', authenticateToken, async (req, res) => {
    try {
        const userID = req.user.userId;
        const [rows] = await database.run({
            sql: `
                SELECT *
                FROM Conversations
                WHERE CreatorID = @userID OR ApplicantID = @userID
            `,
            params: {
                userID,
            },
        });

        if (rows.length === 0) {
            return res.status(404).json({ message: 'No conversations found' });
        }
        res.status(200).json(rows.map(row => row.toJSON()));
    } catch (err) {
        console.error('Error fetching conversations:', err);
        res.status(500).json({ message: 'Failed to fetch conversations' });
    }
});

// GET: Getting messages for a specific conversation and fetch sender names from the Users database
app.get('/api/conversations/:conversationID/messages', authenticateToken, async (req, res) => {
  const conversationID = req.params.conversationID;

  try {
```

```
    // GEtting messages for the conversation
    const query = {
      sql: `SELECT * FROM Messages WHERE ConversationID = @conversationID ORDER BY Timestamp ASC`,
      params: { conversationID },
    };
    const [rows] = await database.run(query);

    if (rows.length === 0) {
      return res.status(404).json({ message: 'No messages found for this conversation' });
    }

    // Getting sender name to display
    const messagesWithSenderNames = await Promise.all(rows.map(async (message) => {
      const senderID = message.find(field => field.name === 'SenderID')?.value;

      if (!senderID) {
        console.error('SenderID is undefined for message:', message);
        return { ...message, SenderName: 'Unknown' };
      }

      try {
        const [userRows] = await usersDatabase.run({
          sql: `SELECT name FROM users WHERE id = @senderID`,
          params: { senderID },
        });
        const senderName = userRows.length > 0 && userRows[0][0]?.value ? userRows[0][0].value : 'Unknown';

        return {
          ...message.toJSON(),
          SenderName: senderName,
        };
      } catch (err) {
        console.error(`Failed to fetch user data for senderID: ${senderID}`, err);
        return { ...message.toJSON(), SenderName: 'Unknown' };
      }
    }));

    res.json(messagesWithSenderNames);
  } catch (err) {
    console.error('Error fetching messages:', err);
    res.status(500).json({ message: 'Failed to fetch messages' });
  }
});



// POST: Send a message in a conversation
app.post('/api/conversations/:conversationID/messages', authenticateToken, async (req, res) => {
  const conversationID = req.params.conversationID;
  const { senderID, messageContent } = req.body;
  const messageID = uuidv4();
```

```
  const timestamp = new Date().toISOString();

  if (!senderID || !messageContent) {
    return res.status(400).json({ message: 'Sender and message content are required' });
  }

  try {
    await database.runTransactionAsync(async (transaction) => {
      const query = {
        sql: `INSERT INTO Messages (MessageID, ConversationID, SenderID, MessageContent, Timestamp)
              VALUES (@messageID, @conversationID, @senderID, @messageContent, @timestamp)`,
        params: {
          messageID,
          conversationID,
          senderID,
          messageContent,
          timestamp,
        },
      };

      await transaction.runUpdate(query);
      await transaction.commit();
      const conversationQuery = {
        sql: `SELECT ApplicantID, CreatorID FROM Conversations WHERE ConversationID = @conversationID`,
        params: { conversationID },
      };

      const [conversationRows] = await database.run(conversationQuery);
      if (conversationRows.length === 0) {
        return res.status(404).json({ message: 'Conversation not found' });
      }

      const conversation = conversationRows[0].toJSON();
      const recipientID = (conversation.ApplicantID === senderID) ? conversation.CreatorID :
conversation.ApplicantID;

      console.log('Recipient ID:', recipientID);

      // Getting the recipient email
      const [userRows] = await usersDatabase.run({
        sql: `SELECT email FROM users WHERE id = @recipientID`,
        params: { recipientID },
      });

      console.log('Result from users table:', userRows);

      const recipientEmail = userRows.length > 0 && userRows[0][0].value ? userRows[0][0].value : null;

      if (!recipientEmail) {
        console.log('No email found for recipient:', recipientID);
        return res.status(400).json({ message: `Recipient email not found for user: ${recipientID}` });
```

```
    }

    const subject = 'You received a new message';
    const emailText = `You received a new message from user ${senderID}: ${messageContent}`;
    await sendEmailNotification(recipientEmail, subject, emailText);

    res.status(200).json({ message: 'Message sent successfully!', messageID });
  });
 } catch (err) {
  console.error('Error sending message:', err);
  res.status(500).json({ message: 'Failed to send message' });
 }
});



// Start the messaging service
const PORT = process.env.PORT || 5004;
app.listen(PORT, () => {
 console.log(`Messaging service is running on port ${PORT}`);
});
```

Search-service/index.js

```
const express = require('express');
const cors = require('cors');
const bodyParser = require('body-parser');
const { Spanner } = require('@google-cloud/spanner');
const jwt = require('jsonwebtoken');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(bodyParser.json());

const spanner = new Spanner({
 projectId: process.env.SPANNER_PROJECT_ID,
});

const instance = spanner.instance(process.env.SPANNER_INSTANCE_ID);
const database = instance.database(process.env.SPANNER_DATABASE_ID);

// Middleware to check JWT and authenticate user
function authenticateToken(req, res, next) {
 const token = req.headers['authorization'];
 if (!token) {
  return res.status(401).json({ message: 'Authorization token is required' });
 }

 jwt.verify(token.split(' ')[1], process.env.JWT_SECRET, (err, user) => {
  if (err) return res.sendStatus(403);
```

```
    req.user = user;
    next();
  });
}

// Endpoint for getting all opportunities
app.get('/api/opportunities', async (req, res) => {
  try {
    const query = {
      sql: 'SELECT OpportunityID, Title, Description, Deadline FROM Opportunities',
    };
    const [rows] = await database.run(query);

    res.status(200).json(rows.map(row => row.toJSON()));
  } catch (err) {
    console.error('Error retrieving opportunities:', err);
    res.status(500).json({ message: 'Error retrieving opportunities' });
  }
});


// Endpoint for Opportunity searching
app.get('/api/opportunities/search', async (req, res) => {
  const { title, location, educationLevel, subjectFilters } = req.query;

  let sql = `SELECT OpportunityID, Title, Description, Location, Deadline, PostedBy, EducationLevel,
SubjectFilters
            FROM Opportunities WHERE 1=1`;

  const params = {};
  //Filters:
  if (title) {
    sql += ` AND LOWER(Title) LIKE LOWER(@title)`;
    params.title = `%${title}%`;
  }
  if (location) {
    sql += ` AND LOWER(Location) LIKE LOWER(@location)`;
    params.location = `%${location}%`;
  }
  if (educationLevel) {
    sql += ` AND EducationLevel = @educationLevel`;
    params.educationLevel = educationLevel;
  }
  if (subjectFilters) {
    const subjectsArray = subjectFilters.split(',').map(s => s.trim());
    sql += ` AND EXISTS (SELECT subject FROM UNNEST(SubjectFilters) AS subject WHERE subject IN
UNNEST(@subjectFilters))`;
    params.subjectFilters = subjectsArray;
  }

  try {
```

```
    const [rows] = await database.run({
      sql,
      params,
    });

    if (rows.length === 0) {
      return res.status(404).json({ message: 'No matching opportunities found' });
    }

    const opportunities = rows.map(row => row.toJSON());
    res.json(opportunities);
  } catch (error) {
    console.error('Search error:', error);
    res.status(500).json({ message: 'Failed to find opportunities' });
  }
});


// Endpoint for Opportunity details
app.get('/api/opportunities/:opportunityID', authenticateToken, async (req, res) => {
  const { opportunityID } = req.params;

  try {
    const query = {
      sql: `SELECT OpportunityID, Title, Description, Location, Deadline, EducationLevel, SubjectFilters, PostedBy
            FROM Opportunities WHERE OpportunityID = @opportunityID`,
      params: {
        opportunityID,
      },
    };

    const [rows] = await database.run(query);
    if (rows.length === 0) {
      return res.status(404).json({ message: 'Opportunity not found' });
    }

    const opportunity = rows[0].toJSON();
    res.status(200).json(opportunity);
  } catch (error) {
    console.error('Error fetching opportunity:', error);
    res.status(500).json({ message: 'Failed to fetch opportunity details' });
  }
});

// Start the server
const PORT = process.env.PORT || 5003;
app.listen(PORT, () => {
  console.log(`Search service is running on port ${PORT}`);
});
```

Opportunity-service/index.js

```
const express = require('express');
const cors = require('cors');
const bodyParser = require('body-parser');
const jwt = require('jsonwebtoken');
const { Spanner } = require('@google-cloud/spanner');
const { v4: uuidv4 } = require('uuid');
const axios = require('axios');
const multer = require('multer');
const { Storage } = require('@google-cloud/storage');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(bodyParser.json());

const spanner = new Spanner({
  projectId: process.env.SPANNER_PROJECT_ID,
});

const instance = spanner.instance(process.env.SPANNER_INSTANCE_ID);
const database = instance.database(process.env.SPANNER_DATABASE_ID);

const storage = new Storage();
const bucket = storage.bucket(process.env.BUCKET_NAME);

const upload = multer({ //creating file storage
  storage: multer.memoryStorage(),
  limits: { fileSize: 10 * 1024 * 1024 },
});

// Middleware to check JWT
async function authenticateToken(request, response, next) {
  const token = request.headers['authorization'];
  if (!token) {
    return response.status(401).json({ message: 'Authorization token is required' });
  }

  try {
    const response = await axios.get(`${process.env.USER_SERVICE_URL}/api/auth/verify`, {
      headers: {
        Authorization: token,
      },
    });
    request.user = response.data;
    next();
  } catch (error) {
    console.error('Token verification failed:', error);
    response.status(403).json({ message: 'Invalid or expired token' });
  }
}
```

```
// POST: Create an Opportunity
app.post('/api/opportunities', authenticateToken, async (request, response) => {
  const { title, description, location, deadline, educationLevel, subjectFilters } = request.body;
  const opportunityID = uuidv4();

  if (!title || !description || !deadline) {
    return response.status(400).json({ message: 'Title, description, and deadline are required' });
  }

  try {
    await database.runTransactionAsync(async (transaction) => {
      const query = {
        sql: `INSERT INTO Opportunities (OpportunityID, Title, Description, Location, Deadline, PostedBy,
EducationLevel, SubjectFilters)
              VALUES (@opportunityID, @title, @description, @location, @deadline, @postedBy, @educationLevel,
@subjectFilters)`,
        params: {
          opportunityID,
          title,
          description,
          location,
          deadline,
          postedBy: request.user.userId,
          educationLevel,
          subjectFilters,
        },
      };

      const rowCount = await transaction.runUpdate(query);
      await transaction.commit();
      response.status(200).json({ message: 'Opportunity posted successfully!', opportunityID });
    });
  } catch (err) {
    console.error('Transaction ERROR:', err);
    response.status(500).json({ error: 'Failed to post opportunity' });
  }
});

// GET: Get all opportunities posted by a specific user
app.get('/api/opportunities', authenticateToken, async (request, response) => {
  try {
    const query = {
      sql: `SELECT * FROM Opportunities WHERE PostedBy = @postedBy`,
      params: { postedBy: request.user.userId },
    };

    const [rows] = await database.run(query);
    response.json(rows.map(row => row.toJSON()));
  } catch (error) {
    console.error('Fetch ERROR:', error);
```

```
    response.status(500).json({ message: 'Failed to fetch opportunities' });
  }
});

// GET: Get a specific opportunity
app.get('/api/opportunities/:id', authenticateToken, async (req, res) => {
  const opportunityID = req.params.id;
  try {
    const query = {
      sql: `SELECT * FROM Opportunities WHERE OpportunityID = @opportunityID`,
      params: { opportunityID },
    };

    const [rows] = await database.run(query);
    if (rows.length === 0) {
      return res.status(404).json({ message: 'Opportunity not found' });
    }
//showing opportunity to every user
    res.json(rows[0].toJSON());
  } catch (error) {
    console.error('Fetch ERROR:', error);
    res.status(500).json({ message: 'Failed to fetch opportunity details' });
  }
});

// PUT: Update an opportunity
app.put('/api/opportunities/:id', authenticateToken, async (req, res) => {
  const opportunityID = req.params.id;
  const { title, description, location, deadline, educationLevel, subjectFilters } = req.body;

  if (!title || !description || !deadline) {
    return res.status(400).json({ message: 'Title, description, and deadline are required' });
  }

  try {
    await database.runTransactionAsync(async (transaction) => {
      const query = {
        sql: `UPDATE Opportunities
              SET Title = @title, Description = @description, Location = @location,
                  Deadline = @deadline, EducationLevel = @educationLevel, SubjectFilters = @subjectFilters
              WHERE OpportunityID = @opportunityID AND PostedBy = @postedBy`,
        params: {
          opportunityID,
          title,
          description,
          location,
          deadline,
          postedBy: req.user.userId,
          educationLevel,
          subjectFilters,
        },
```

```
      };

      const rowCount = await transaction.runUpdate(query);
      await transaction.commit();
      res.status(200).json({ message: 'Opportunity updated successfully!' });
    });
  } catch (err) {
    console.error('Transaction ERROR:', err);
    res.status(500).json({ error: 'Failed to update opportunity' });
  }
});

// DELETE: Delete an opportunity
app.delete('/api/opportunities/:id', authenticateToken, async (req, res) => {
  const opportunityID = req.params.id;

  try {
    await database.runTransactionAsync(async (transaction) => {
      const query = {
        sql: `DELETE FROM Opportunities WHERE OpportunityID = @opportunityID AND PostedBy =
@postedBy`,
        params: { opportunityID, postedBy: req.user.userId },
      };

      const rowCount = await transaction.runUpdate(query);
      await transaction.commit();
      res.status(200).json({ message: 'Opportunity deleted successfully!' });
    });
  } catch (err) {
    console.error('Transaction ERROR:', err);
    res.status(500).json({ error: 'Failed to delete opportunity' });
  }
});


// POST: Apply for an opportunity
app.post('/api/opportunities/:id/apply', authenticateToken, upload.single('cv'), async (req, res) => {
  const { motivationLetter } = req.body;
  const opportunityID = req.params.id;
  const applicationID = uuidv4();
  let cvUrl = null;

  if (!motivationLetter) {
    return res.status(400).json({ message: 'Motivation letter is required' });
  }

  if (req.file) {
    try {
      const blob = bucket.file(`cvs/${applicationID}-${req.file.originalname}`);
      const blobStream = blob.createWriteStream({
        resumable: false,
```

```
      metadata: {
        contentType: req.file.mimetype,
      },
    });

    blobStream.on('error', (err) => {
      console.error('Error uploading file to GCS:', err);
      res.status(500).json({ message: 'Failed to upload CV' });
    });

    blobStream.on('finish', () => {
      cvUrl = `https://storage.cloud.google.com/${bucket.name}/${blob.name}`;
      console.log('File uploaded to GCS:', cvUrl);
      submitApplication();
    });

    blobStream.end(req.file.buffer); // Uploadign the file buffer to GCS
  } catch (err) {
    console.error('Error handling file upload:', err);
    return res.status(500).json({ message: 'Failed to process CV upload' });
  }
 } else {
  submitApplication();
 }

 // Function to handle application submission
 async function submitApplication() {
  try {
    await database.runTransactionAsync(async (transaction) => {
      const query = {
        sql: `INSERT INTO Applications (ApplicationID, OpportunityID, UserID, ApplicationDate, MotivationLetter,
CVUrl)
              VALUES (@applicationID, @opportunityID, @userID, PENDING_COMMIT_TIMESTAMP(),
@motivationLetter, @cvUrl)`,
        params: {
          applicationID,
          opportunityID,
          userID: req.user.userId,
          motivationLetter,
          cvUrl,
        },
      };

      await transaction.runUpdate(query);
      await transaction.commit();
      res.status(200).json({ message: 'Application submitted successfully!', applicationID });
    });
  } catch (err) {
    console.error('Transaction ERROR:', err);
    res.status(500).json({ error: 'Failed to submit application' });
  }
```

```
  }
});

// GET: Get all applications for a user
app.get('/api/my-applications', authenticateToken, async (req, res) => {
  try {
    const query = {
      sql: `SELECT * FROM Applications WHERE UserID = @userID`,
      params: { userID: req.user.userId },
    };

    const [rows] = await database.run(query);
    res.json(rows.map(row => row.toJSON()));
  } catch (error) {
    console.error('Fetch ERROR:', error);
    res.status(500).json({ message: 'Failed to fetch applications' });
  }
});

// GET: Get all applications for an opportunity
app.get('/api/opportunities/:id/applications', authenticateToken, async (req, res) => {
  const opportunityID = req.params.id;

  try {
    const query = {
      sql: `SELECT * FROM Applications WHERE OpportunityID = @opportunityID`,
      params: { opportunityID },
    };

    const [rows] = await database.run(query);
    res.json(rows.map(row => row.toJSON()));
  } catch (error) {
    console.error('Fetch ERROR:', error);
    res.status(500).json({ message: 'Failed to fetch applications for opportunity' });
  }
});

module.exports = app;
module.exports = { authenticateToken };

// Start the server
const PORT = process.env.PORT || 5002;
app.listen(PORT, () => {
  console.log(`Opportunity service is running on port ${PORT}`);
});
```

user-service/__tests__/authService.test.js

```
const bcrypt = require('bcrypt');

describe('Password Hashing', () => {
  it('should hash a password correctly', async () => {
```

```
    const password = 'password123';
    const hashedPassword = await bcrypt.hash(password, 10);
    expect(hashedPassword).not.toBe(password);
    const isValid = await bcrypt.compare(password, hashedPassword);
    expect(isValid).toBe(true);
  });
});
```

backend/user-service/__tests__/jwtService.test.js

```
const jwt = require('jsonwebtoken');

describe('JWT Token Generation', () => {
  it('should generate a valid token for a user', () => {
    const userId = 'user-123';
    const secret = 'test-secret';

    const token = jwt.sign({ userId }, secret, { expiresIn: '1h' });
    const decoded = jwt.verify(token, secret);
    expect(decoded.userId).toBe(userId);
  });

  it('should throw an error for an invalid token', () => {
    const invalidToken = 'invalid-token';
    const secret = 'test-secret';

    expect(() => jwt.verify(invalidToken, secret)).toThrow();
  });
});
```

User-service/authService.js

```
const { Spanner } = require('@google-cloud/spanner');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const { OAuth2Client } = require('google-auth-library');
const uuid = require('uuid');
require('dotenv').config();

const spanner = new Spanner({
  projectId: process.env.SPANNER_PROJECT_ID,
});

const instance = spanner.instance(process.env.SPANNER_INSTANCE_ID);
const database = instance.database(process.env.SPANNER_DATABASE_ID);

const googleClient = new OAuth2Client(process.env.GOOGLE_CLIENT_ID);

async function register(name, email, password) {
  const hashedPassword = await bcrypt.hash(password, 10);
```

```
  const userId = uuid.v4();

  const query = {
    sql: `INSERT INTO users (id, name, email, password) VALUES (@id, @name, @Email, @password)`,
    params: {
      id: userId,
      name,
      email,
      password: hashedPassword,
    },
  };

  await database.run(query);
  return userId;
}

async function login(email, password) {
  const query = {
    sql: `SELECT id, password FROM users WHERE email = @Email`,
    params: { email },
  };

  const [rows] = await database.run(query);
  if (rows.length === 0) {
    throw new Error('User not found');
  }

  const user = rows[0].toJSON();
  const isPasswordValid = await bcrypt.compare(password, user.password);
  if (!isPasswordValid) {
    throw new Error('Invalid password');
  }

  const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET);
  return token;
}

async function googleLogin(tokenId) {
  const ticket = await googleClient.verifyIdToken({
    idToken: tokenId,
    audience: process.env.GOOGLE_CLIENT_ID,
  });
  const payload = ticket.getPayload();

  const query = {
    sql: `SELECT id FROM users WHERE googleId = @googleId`,
    params: { googleId: payload.sub },
  };

  const [rows] = await database.run(query);
  let userId;
```

```
  if (rows.length === 0) {
    userId = uuid.v4();
    const insertQuery = {
      sql: `INSERT INTO users (id, name, email, googleId) VALUES (@id, @name, @Email, @googleId)`,
      params: {
        id: userId,
        name: payload.name,
        email: payload.email,
        googleId: payload.sub,
      },
    };
    await database.run(insertQuery);
  } else {
    userId = rows[0].toJSON().id;
  }

  const token = jwt.sign({ userId }, process.env.JWT_SECRET);
  return token;
}

module.exports = {
  register,
  login,
  googleLogin,
};
```

User-service/index.js

```
const express = require('express');
const bodyParser = require('body-parser');
const cors = require('cors');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
const { register, login, googleLogin } = require('./authService');
const { Spanner } = require('@google-cloud/spanner');
const { v4: uuidv4 } = require('uuid');
require('dotenv').config();

const app = express();
const PORT = process.env.PORT || 5001;

app.use(bodyParser.json());
app.use(cors());

const spanner = new Spanner({
  projectId: process.env.SPANNER_PROJECT_ID,
});

const instance = spanner.instance(process.env.SPANNER_INSTANCE_ID);
const database = instance.database(process.env.SPANNER_DATABASE_ID);
```

```
// Middleware to check JWT
function authenticateToken(req, res, next) {
  const token = req.headers['authorization'];
  if (!token) return res.sendStatus(401);

  jwt.verify(token.split(' ')[1], process.env.JWT_SECRET, (err, user) => {
    if (err) return res.sendStatus(403);
    req.user = user;
    next();
  });
}

// Registration endpoint
app.post('/api/auth/register', async (req, res) => {
  const { name, email, password } = req.body;

  try {
    const userId = uuidv4();
    const hashedPassword = await bcrypt.hash(password, 10);

    await database.runTransactionAsync(async (transaction) => {
      const query = {
        sql: `INSERT INTO users (id, name, email, password)
              VALUES (@id, @name, @email, @password)`,
        params: {
          id: userId,
          name: name,
          email: email,
          password: hashedPassword,
        },
      };

      await transaction.runUpdate(query);
      await transaction.commit();
    });

    const token = jwt.sign({ userId }, process.env.JWT_SECRET, { expiresIn: '1h' });
    res.json({ token });
  } catch (error) {
    console.error('Registration error:', error);
    res.status(400).json({ message: error.message });
  }
});

// Login endpoint
app.post('/api/auth/login', async (req, res) => {
  const { email, password } = req.body;

  try {
    const query = {
      sql: `SELECT id, name, email, password FROM users WHERE email = @email`,
```

```
      params: { email },
    };

    const [rows] = await database.run(query);
    if (rows.length === 0) {
      return res.status(400).json({ message: 'Invalid email or password' });
    }
    const user = rows[0].toJSON();
    const isPasswordValid = await bcrypt.compare(password, user.password);
    if (!isPasswordValid) {
      return res.status(400).json({ message: 'Invalid email or password' });
    }

    // Generation of a a JWT token
    const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET, { expiresIn: '12h' });
    res.json({ token });
  } catch (error) {
    console.error('Login error:', error);
    res.status(400).json({ message: error.message });
  }
});

// Google login endpoint
app.post('/api/auth/google', async (req, res) => {
  try {
    const { tokenId } = req.body;
    const token = await googleLogin(tokenId);
    res.json({ token });
  } catch (error) {
    console.error('Google login error:', error);
    res.status(400).json({ message: error.message });
  }
});

// User profile endpoint
app.get('/api/auth/profile', authenticateToken, async (req, res) => {
  try {
    const query = {
      sql: `SELECT id, name, email FROM users WHERE id = @id`,
      params: { id: req.user.userId },
    };

    const [rows] = await database.run(query);
    if (rows.length === 0) {
      return res.status(404).json({ message: 'User not found' });
    }

    res.json(rows[0].toJSON());
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
```

```
});

// Token verification endpoint
app.get('/api/auth/verify', authenticateToken, (req, res) => {
  res.json(req.user);
});

// Start the server
app.listen(PORT, () => {
  console.log(`User service is running on port ${PORT}`);
});
```